NoSQL Database Systems

M.Tech. (IInd, Sem CE/CN)

Datasets

Collections or groups of related data are generally referred to as datasets. Each group or dataset member (datum) shares the same set of attributes or properties as others in the same dataset. Some examples of datasets are:

- tweets stored in a flat file
- a collection of image files in a directory
- an extract of rows from a database table stored in a CSV formatted file
- historical weather observations that are stored as XML files

Data Analysis

Data analysis is the process of examining data to find facts, relationships, patterns, insights and/or trends. The overall goal of data analysis is to support better decision making. A simple data analysis example is :

1. The analysis of ice cream sales data in order to determine how the number of ice cream cones sold is related to the daily temperature. The results of such an analysis would support decisions related to
   - how much ice cream a store should order in relation to weather forecast information.

Carrying out data analysis helps establish patterns and relationships among the data being analyzed

Data Analytics

- Data analytics is a broader term that encompasses data analysis. Data analytics is a discipline that includes the management of the complete data lifecycle, which encompasses
  - collecting, cleansing, organizing, storing, analyzing and governing data.

- The term includes the development of analysis methods, scientific techniques and automated tools.
- In Big Data environments, data analytics has developed methods that allow data analysis to occur through the use of highly scalable distributed technologies and frameworks that are capable of analyzing large volumes of data from different sources.

Types of Data Analytics

There are four general categories of analytics that are distinguished by the results they produce:

- descriptive analytics
- diagnostic analytics
- predictive analytics
- prescriptive analytics

Descriptive Analytics

- Descriptive analytics are carried out to answer questions about events that have already occurred. This form of analytics contextualizes data to generate information.

Sample questions can include:

- What was the sales volume over the past 12 months?
- What is the number of support calls received as categorized by severity and geographic location?
- What is the monthly commission earned by each sales agent?

It is estimated that 80% of generated analytics results are descriptive in nature. Value wise, descriptive analytics provide the least worth and require a relatively basic skill set.

Descriptive Analytics

- Descriptive analytics are often carried out via ad-hoc reporting or dashboards.
- The reports are generally static in nature and display historical data that is presented in the form of data grids or charts.
- Queries are executed on operational data stores from within an enterprise, for example a Customer Relationship Management system (CRM) or Enterprise Resource Planning (ERP) system.

Diagnostic Analytics

Diagnostic analytics aim to determine the cause of a phenomenon that occurred in the past using questions that focus on the reason behind the event. The goal of this type of analytics is to determine what information is related to the phenomenon in order to enable answering questions that seek to determine why something has occurred.

Such questions include:

- Why were Q2 sales less than Q1 sales?
- Why have there been more support calls originating from the Eastern region than from the Western region?
- Why was there an increase in patient re-admission rates over the past three months?

Diagnostics Analytics

- Diagnostic analytics provide more value than descriptive analytics but require a more advanced skill set.
- Diagnostic analytics usually require collecting data from multiple sources and storing it in a structure that lends itself to performing drill-down and roll-up analysis.
- Diagnostic analytics results are viewed via interactive visualization tools that enable users to identify trends and patterns.
- The executed queries are more complex compared to those of descriptive analytics and are performed on multidimensional data held in analytic processing systems.

Predictive Analytics

- Predictive analytics are carried out in an attempt to determine the outcome of an event that might occur in the future. With predictive analytics, information is enhanced with meaning to generate knowledge that conveys how that information is related.
- The strength and magnitude of the associations form the basis of models that are used to generate future predictions based upon past events.
- It is important to understand that the models used for predictive analytics have implicit dependencies on the conditions under which the past events occurred. If these underlying conditions change, then the models that make predictions need to be updated.

Predictive Analytics

Questions are usually formulated using a what-if rationale, such as the following:

- What are the chances that a customer will default on a loan if they have missed a monthly payment?
- What will be the patient survival rate if Drug B is administered instead of Drug A?
- If a customer has purchased Products A and B, what are the chances that they will also purchase Product C?

Prescriptive Analytics

- Prescriptive analytics build upon the results of predictive analytics by prescribing actions that should be taken.
- The focus is not only on which prescribed option is best to follow, but why. In other words, prescriptive analytics provide results that can be reasoned about because they embed elements of situational understanding.
- Thus, this kind of analytics can be used to gain an advantage or mitigate a risk.

Prescriptive Analytics

- Sample questions may include:
- Among three drugs, which one provides the best results?
- When is the best time to trade a particular stock?

Types of Data

- Structured
- Un-structured
- Semi-structured
- Meta-data

Structured Data

- Structured data conforms to a data model or schema and is often stored in tabular form.
- It is used to capture relationships between different entities and is therefore most often stored in a relational database.
- Structured data is frequently generated by
  - Enterprise applications and information systems like ERP and CRM systems.

Examples of this type of data include banking transactions, invoices, and customer records.

Unstructured Data

- Data that does not conform to a data model or data schema is known as unstructured data.
- It is estimated that unstructured data makes up 80% of the data under the umbrella of Big data.
- Unstructured data has a faster growth rate than structured data.
- This form of data is either textual or binary and often conveyed via files that are self-contained and non-relational.
  - For Example a text file may contain the contents of various tweets or blog postings.
  - Binary files are often media files that contain image, audio or video data.

  Technically, both text and binary files have a structure defined by the file format itself, but this aspect is disregarded, and the notion of being unstructured is in relation to the format of the data contained in the file itself.

Unstructured Data

- Video, image and audio files are all types of unstructured data.
- Special purpose logic is usually required to process and store unstructured data. For example, to play a video file, it is essential that the correct codec (coder-decoder) is available.
- Unstructured data cannot be directly processed or queried using SQL.
- If it is required to be stored within a relational database, it is stored in a table as a Binary Large Object (BLOB)
- Alternatively, a Not-only SQL (NoSQL) database is a non-relational database that can be used to store unstructured data alongside structured data.

Semi-structured data

- Semi-structured data has a defined level of structure and consistency, but is not relational in nature.
- Instead, semi-structured data is hierarchical or graph-based.
- This kind of data is commonly stored in files that contain text.
  - For example XML and JSON files are common forms of semi-structured data.
- Due to the textual nature of this data and its conformance to some level of structure, it is more easily processed than unstructured data.

Semi-structured data

- XML, JSON and sensor data are semi-structured.
- Examples of common sources of semi-structured data include electronic data interchange (EDI) files, spreadsheets, RSS feeds and sensor data.
- Semi-structured data often has special pre-processing and storage requirements, especially if the underlying format is not text-based.
- An example of pre-processing of semi-structured data would be the validation of an XML file to ensure that it conformed to its schema definition.

Metadata

- Metadata provides information about a dataset's characteristics and structure. This type of data is mostly machine-generated and can be appended to data.

- The tracking of metadata is crucial to Big Data processing, storage and analysis because it provides information about the pedigree of the data and its provenance during processing. Examples of metadata include:
  - XML tags providing the author and creation date of a document
    attributes providing the file size and resolution of a digital photograph

- Big Data solutions rely on metadata, particularly when processing semi-structured and unstructured data.

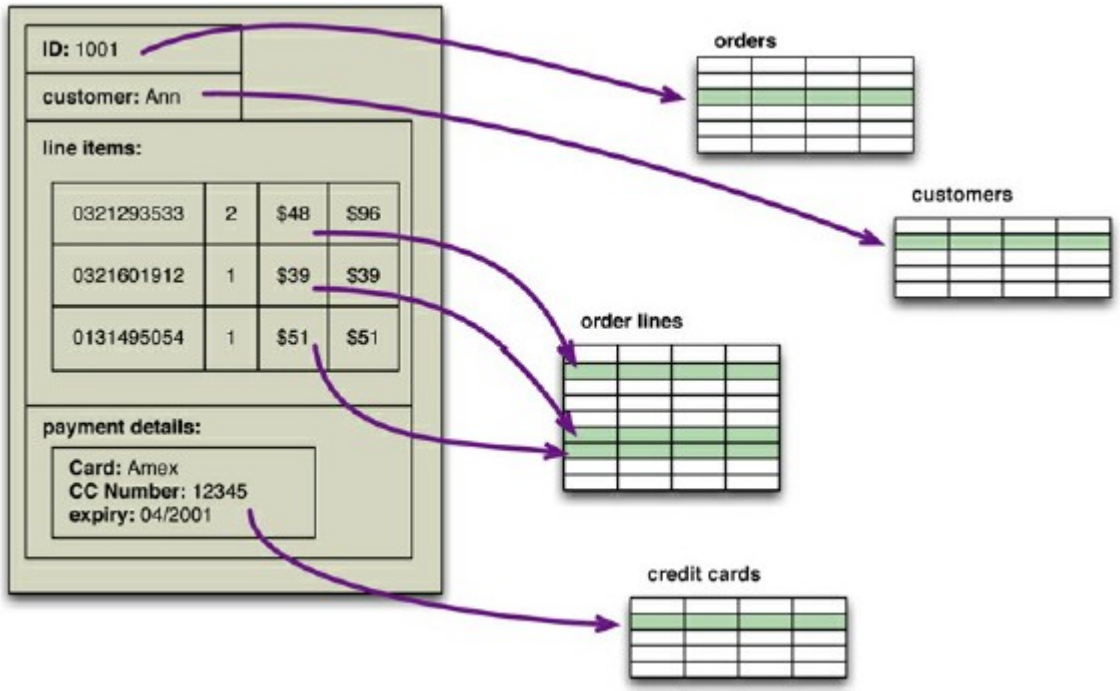
Introduction to NoSQL

- The use of relational database model has been considered to solve almost any problem of data management and storage. Choice of Enterprise. (Persistence, Concurrency, Integration, Standard Model)

Challenges with RDBMs

- Impedance Mismatch
  - Difference between relational model (relationship & tuples) and in-memory data structures
- Un-structured data storage (text, video, tweets, mail etc.)
- Application integration databases to application databases.
- Attack of clusters

Impedance mismatch



Intro : What means NoSQL

- NoSQL was popularized to refer to databases that do not support the relational model, and they do not use the structured query language of relational databases.

- The term NoSQL groups all those technologies, like graphs, documents, text et cetera. That by definition are non-relational.

- A more precise definition of NoSQL corresponds to all those new generation databases that are non-relational, distributed, open source, schema less and horizontally scalable.

- Summarizing, NoSQL, now, not only SQL is a general category of database management system that differs from the relational database management system in different ways. They do not have a schema, they do not allow joins, they do not try to guarantee the transaction ACID properties, and they scale horizontally.

Diff. Between NoSQL & RDBMs

- **Data models** : In the case of data models, a NoSQL database lets you build an application without having to define the schema first unlike relational databases.

- **Data structure** : Regarding data structure, NoSQL databases are designed to handle unstructured data, such as text, social media, post, video, or email, which makes up much of the data that exist today.

- **Development model** : Regarding development model, NoSQL databases are open source, whereas relational databases typically are closed source with licensing fees baked it into the use of their software. With NoSQL, you can get started on a project without any heavy investment in software fees

Cassandra, Ubuntu One, CouchDB

- The NoSQL movement also got the attention of several companies that have developed NoSQL technologies, like Facebook, who developed the Cassandra system, later used by Twitter.

- In addition, Linkedin developed Project Voldemort and Ubuntu One, a synchronized cloud storage system based on CouchDB.

- In addition to web applications, NoSQL databases also support diverse activity, including predictive analysis and non-critical transactional systems, those that do not require the ACID properties.

Benefits of NoSQL

- As data is stored schema-less, Read and Write speeds are much higher in NoSQL as compared to RDBMs. Data is written at greatest speeds in a NoSQL database are usually fast and then provide high performance.
  - For example, Hypertable's technology is capable of storing one trillion data per day.
  - Another example is a Google BigTable software, which can process 20 petabytes of data in a day.
- Other benefit is that if a database experiences some growth, it is possible to add nodes to a distributed system is such a way that it provides processing and storage economically, also known as horizontal scalability.
- NoSQL databases have the necessary mechanisms for horizontal scalability. In terms of a storage, the NoSQL storage model is much simpler than the table schema of the relational model.

More Benefits ...

- Furthermore, NoSQL storage is more flexible than the rigid schema of tables. In reality, NoSQL databases lack a fixed schema. When the data to be stored cannot be translated into tables of the relational model, it is convenient to look for a solution by NoSQL.

- NoSQL databases are a good option for those information systems that do not require the ACID properties. Therefore, without a strict management of a transactions, the control of concurrency is not done by blocking, but there are other mechanisms.

- Furthermore, the principal relational database management systems have absorbed alternative proposals such as XML document warehouses and object-oriented databases.

- Far from replacing the relational database management systems, they complement and extend their functional characteristics.

- Since durability is not a critical element in some web applications, it's not necessary to write logs for each transaction. Examples of data not sensitive to durability include copies of user session data and text message in forums or social networks.

Challenges with NoSQL

- NoSQL has a **non-standardized query language.** In the absence of a standard query language for NoSQL databases, it is necessary to learn each query language for a given database management system. This is true even within each NoSQL storage category.
- NoSQL has **problems in transactions**. As a result of not following the ACID properties, there is a weaker control over the consistency, durability and isolation of transactions.
- NoSQL has **integrity problems**. Ensuring the integrity of the data requires extra programming manually and is not function of the DBMS. Integrity is understood from the point of view of the restrictions, of domain, referential of null value, etc.

RDBMS or NoSQL

The decision on what type of DBMS should be used depends on a set of factors including but not limited to.

- The volume of data to store.
- The estimated concurrence.
- The number of operations that are done on the database per unit of time.
- The desired scalability of the database.
- The degree of integrity and consistency that is desired.
- The nature of the data to be stored.
- The most frequent types of operations that you want to do with the data.

CAP and BASE

There are two main characteristics regarding NoSQL data bases and they are the CAP theorem and the BASE properties. On the one hand, the CAP theorem has been adopted by several companies in the web and the NoSQL community. The acronym for CAP refers to the following,

- **C as consistency**. It refers to whether a system is in a consistent state after the execution of an operation or not. A distributed system is considered consistent if after an update operation by a node, the rest of the nodes see the update in a shared data resource.
- **A as availability**, it means that a system is designed and implemented in such a way that it can continue its operation if there are software or hardware problems or a node fails.
- **P as Partition tolerance** it's the ability of a system to continue it's operation in the presence of network partitions. This happens if a set of nodes in a network looses connectivity with other nodes. Partition tolerance can also be considered as ability of a system to dynamically add and remove nodes.

BASE

So basically the CAP theorem says that the most you can have two of the three characteristics in a shared data system.

- On the other hand, a NoSQL database follows a paradigm of the BASE properties, Basically available system instead of ACID
- Basically available.
- Soft state,

  (the information will expire unless it is refreshed.)
- Eventual consistency.

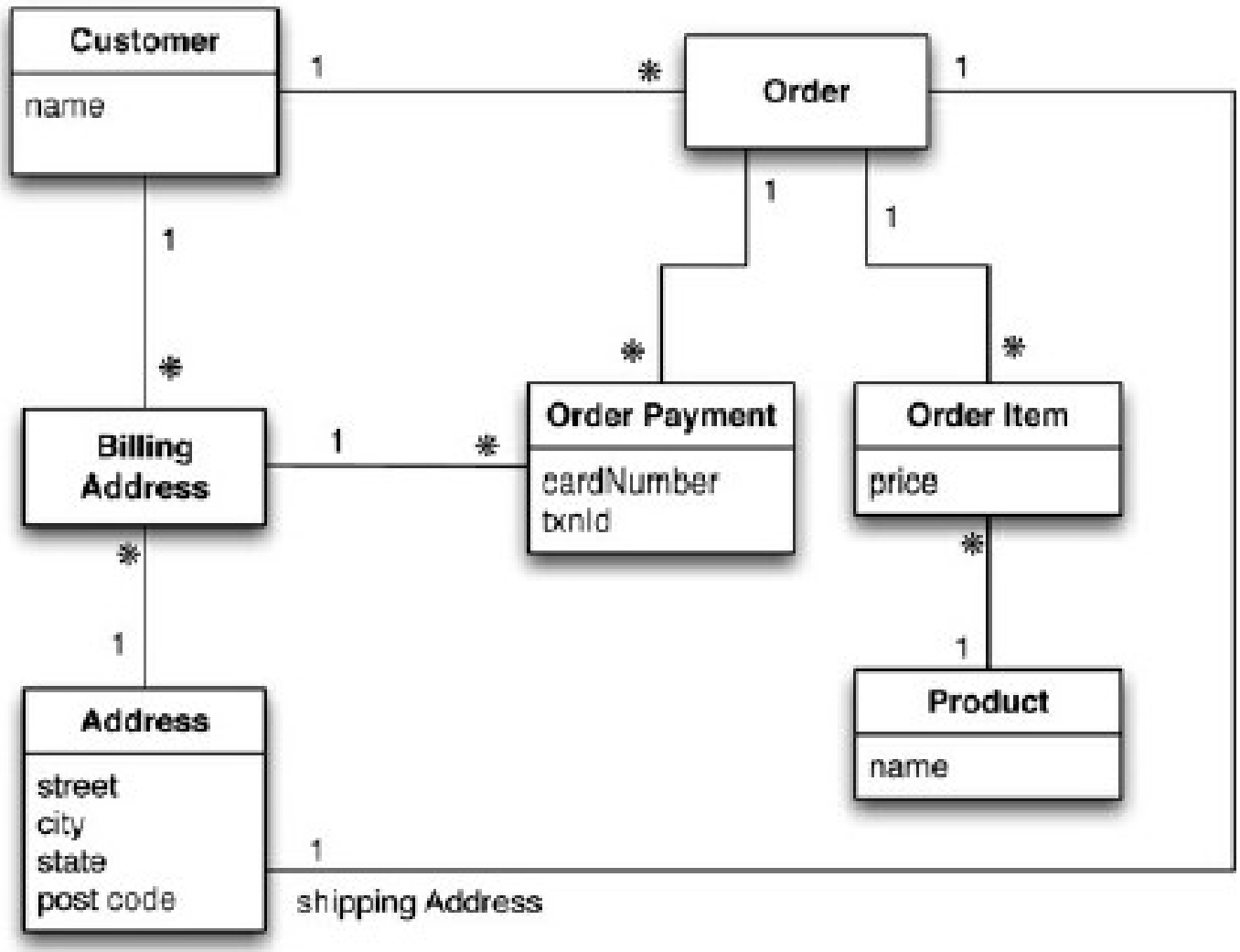
The BASE properties can be summarized as follows,

An application works basically all the time. It does not have to be consistent all the time, but it will eventually reach a known state.

Aggregate Data Models

- A data model is the mechanism through which we perceive and manipulate our data.
- An aggregate data model is a collection of related objects which can be treated together as a unit.
- These data models require aggregates to be updated in atomic operations and get communicated to storage as aggregates.
- Key-Value/Document/Column family databases have aggregate orientation

RDBM Oriented Data Model



Data Representation in RDBMS

Customer	
Id	Name
1	Martin

Orders		
Id	CustomerId	ShippingAddressId
99	1	77

Product	
Id	Name
27	NoSQL Distilled

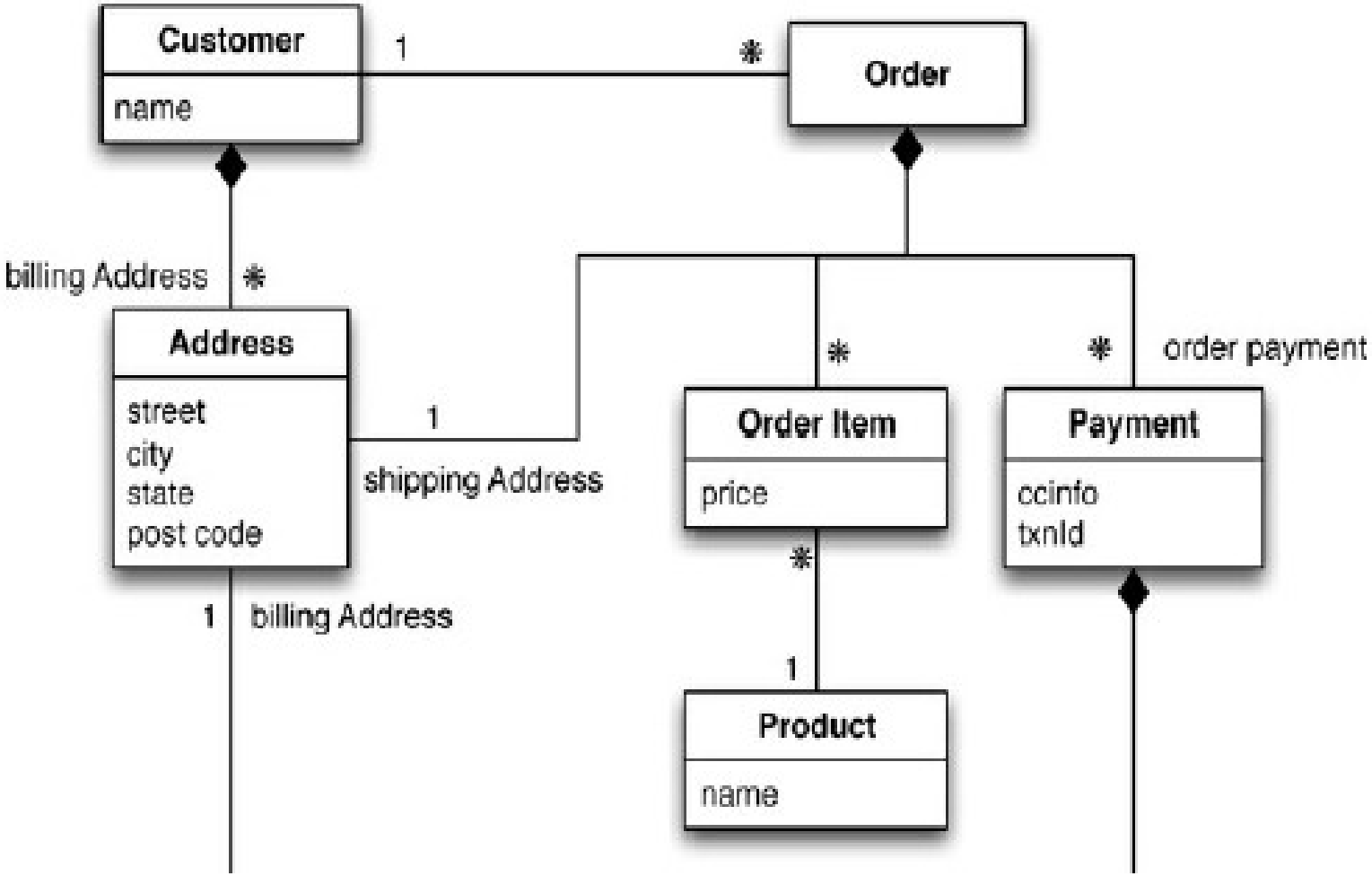
BillingAddress		
Id	CustomerId	AddressId
55	1	77

OrderItem			
Id	OrderId	ProductId	Price
100	99	27	32.45

Address	
Id	City
77	Chicago

OrderPayment				
Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Aggregate Data Model



JASON representation

- // in customers{

"id":1,

"name":"Martin",

"billingAddress":[{"city":"Chicago"}]}

- // in orders{

"id":99,

"customerId":1,

"orderItems":[{

"productId":27,

"price": 32.45,

"productName": "NoSQL Distilled" }],

"shippingAddress":[{"city":"Chicago"}]

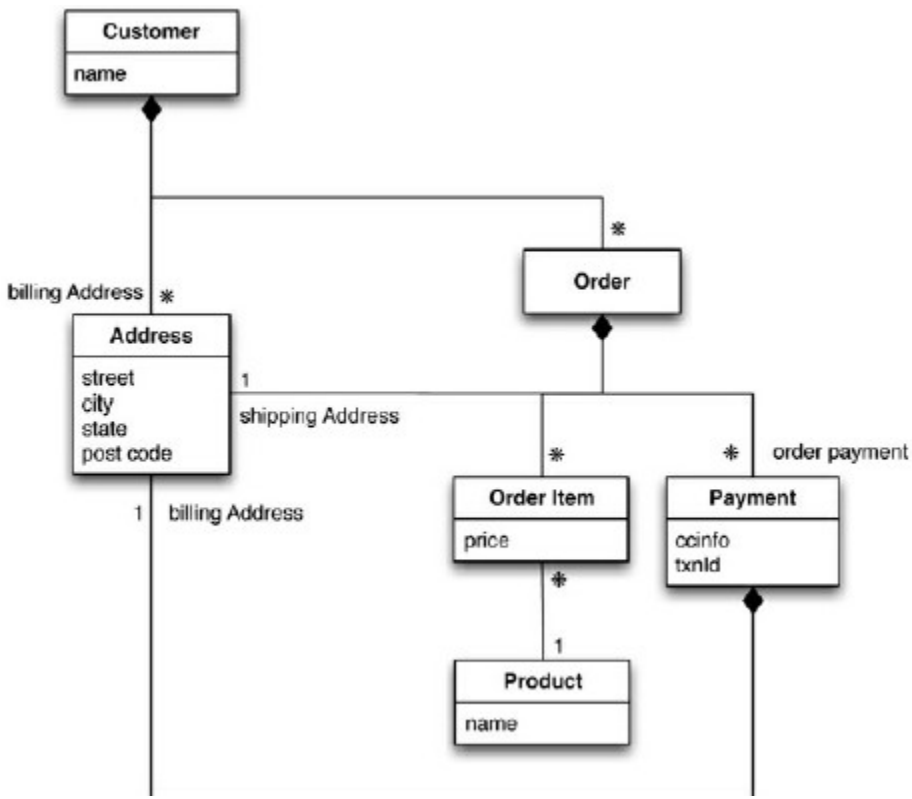
"orderPayment":[{

"ccinfo":"1000-1000-1000-1000",

"txnId":"abelif879rft",

"billingAddress": {"city": "Chicago"} }],}

Customer aggregation containing orders and its aggregates



JASON representation

- // in customers
- “customer”: {

"id":1,
"name":"Martin",
"billingAddress":[{"city":"Chicago"}]},
“orders” [{
"id":99,
"customerId":1,
"orderItems":[{
"productId":27,
"price": 32.45,
"productName": "NoSQL Distilled" }],
"shippingAddress":[{"city":"Chicago"}]
"orderPayment":[{
"ccinfo":"1000-1000-1000-1000",
"txnId":"abelif879rft",
"billingAddress": {"city": "Chicago"} }],}]

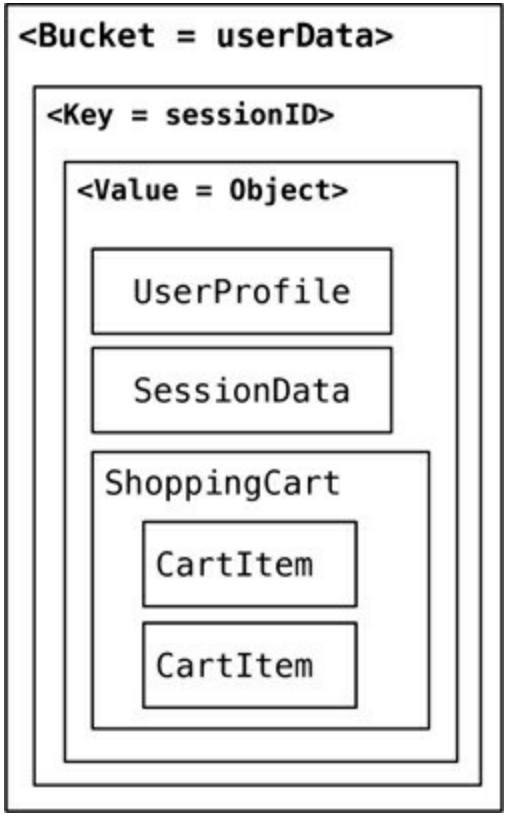
Key-Value database

It is a database system that stores values indexed by keys and it can store structured and unstructured data. (The possibility of storing any type of value is called schema-less.)

- The data values are stored as arrays of bytes.
- The content is not important for the database.
- These databases offer high performance, very scalable, very flexible, and low complexity.

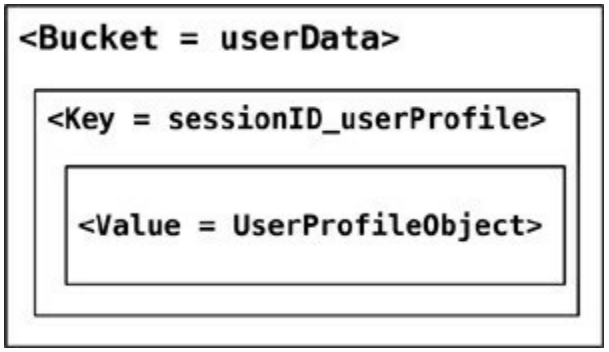
- The disadvantage of key value databases is that they do not support complex queries, because they only look for a key.
- Examples of key-value databases are Amazon, Riak, DynamoDB, Voldemort, RAMCloud, and Flare.

Key-Value Database



Key-Value Database

Domain Specific Key



Document Databases

- Document databases are commonly key value where a value is stored as a binary field with a format that the DBMS can understand.

- This format is often a JSON document (JavaScript Object Notation), BSON (Binary JSON) but it can be XML or any other.
  - These databases allow very advanced queries on the data.
  - Allows relations between data.
  - Do not allow join operations due to performance issues.
- Examples of document-oriented DBMS include CouchDB, MongoDB, Cloudkit, and XML databases such as DB2 pureXML.

Document store

MongoDB saves data in JSON-type documents with a dynamic schema called BSON, which implies that there is no predefined schema.

The elements of the data are called documents and are stored in collections.

- A collection can have an indeterminate number of documents.

Collections are like tables and documents like rows.

Each document in a collection can have different fields.

- The structure of a document is simple and composed of "key-value pairs" similar to the associative matrices in a programming language.

As a value you can use numbers, strings or binary data as images or any other "key-value pairs".

Document store

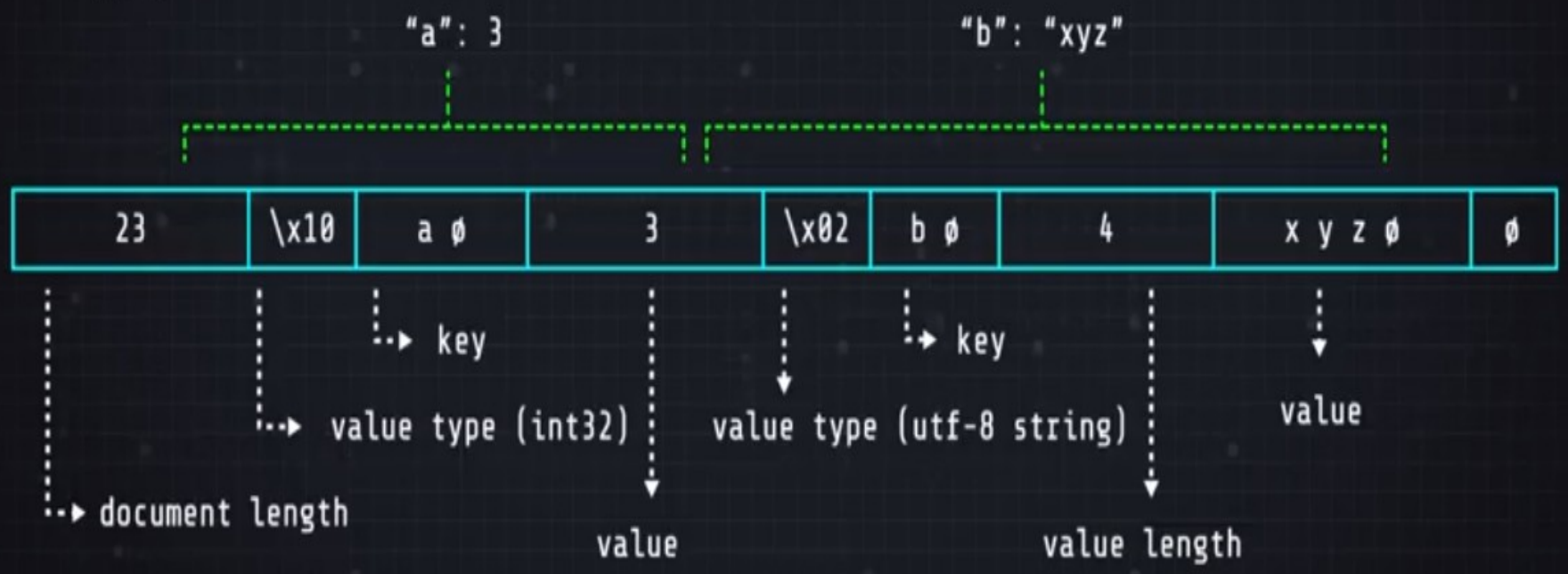
```
{
    name: "sue",                          <------- field: value
    age: 26,                              <------- field: value
    status: "A",                          <------- field: value
    groups: [ "news", "sports"]           <------- field: value
}
```

BSON representation

JASON

```
{
"a": 3,
"b": "xyz"
}
```

BSON



Document store

Documents are stored in collections, which are a set of related documents that share indexes. The collections are analogous to the tables of the relational databases. The figure conceptually shows a collection of MongoDB.

Document storage offers high performance, variable scalability, high flexibility and low implementation complexity.

ObjectId type

_id is the first attribute of the document and the internal identifier in hexadecimal in this example

```
{ "_id": ObjectId("4efa8d2b7d284dad101e4bc7"),
   "Last Name": "PELLERIN",
   "First Name": "Frank",
   "Age": 29,
   "Address": {
      "Street": "1 chemin des Loges",
      "City": "VERSAILLES"
   }
}
```

Samples

SQL

```
CREATE TABLE users (name VARCHAR(128), age
NUMBER)
INSERT INTO users VALUES ('Bob', 32)
SELECT * FROM users
SELECT name, age FROM users
SELECT name, age FROM users WHERE age = 33

SELECT * FROM users WHERE age > 33
SELECT * FROM users WHERE age <= 33
```

mongoDB

```
db.createCollection("users)

db.users.insert({name: "Bob", age: 32})
db.users.find()
db.users.find({}, {name: 1, age: 1, _id:0})
db.users.find({age: 33}, {name: 1, age: 1,
_id:0})
db.users.find({age: {$gt: 33}})
db.users.find({age: {$lte: 33}})
```

SQL

```
SELECT * FROM users LIMIT 1
SELECT DISTINCT name FROM users
SELECT COUNT (*) FROM users
SELECT COUNT (*) FROM users WHERE AGE > 30
SELECT COUNT(AGE) FROM users

UPDATE users SET age = 33 WHERE name =
'Bob'
UPDATE users SET age = 33 + 2 WHERE name
= 'Bob'
DELETE FROM users WHERE name = 'Bob'
CREATE INDEX ON users (name ASC)
CREATE INDEX ON users (name ASC, age DESC)
EXPLAIN SELECT * FROM users WHERE age = 32
```

mongoDB

```
db.users.finOne()
db.users.distinct("name")
db.users.count()
db.users.find({age: {$gt: 30}}).count()
db.users.find({age: {$exist: true}}).
count()
db.users.update({name: "Bob"}, {$set:
{age: 33}}, {multi: true}"
db.users.update({name: "Bob"}, {$inc:
{age: 2}}, {multi: true}"
db.users.remove({name: "Bob"})
db.users.ensureIndex({name: 1})
db.users.ensureIndex({name: 1, age: -1})
db.users.find({age: 32}).explain()
```

Document Databases

```
{
"firstname": "Martin",
"likes": [ "Biking",
"Photography" ],
"lastcity": "Boston",
"lastVisited":
}
```

The above shown document may be represented as a row in RDBMS.

```
{
"firstname": "Pramod",
"citiesvisited": [ "Chicago", "London", "Pune", "Bangalore" ],
"addresses": [
{ "state": "AK",
"city": "DILLINGHAM", "type": "R"
},
{ "state": "MH", "city": "PUNE", "type": "R" }
],
"lastcity": "Chicago"
}
```

Column Family Database

- Regarding the column-oriented databases, as the name implies, they store the data in columns instead of rows.
- That is, all the attributes of a single data entity are stored so that each of them can be accessed as a unit.
- Examples of DBMS oriented to columns includes Sybase IQ, Cassandra, hyper table, and HBase

Column-oriented databases

What is a column store?

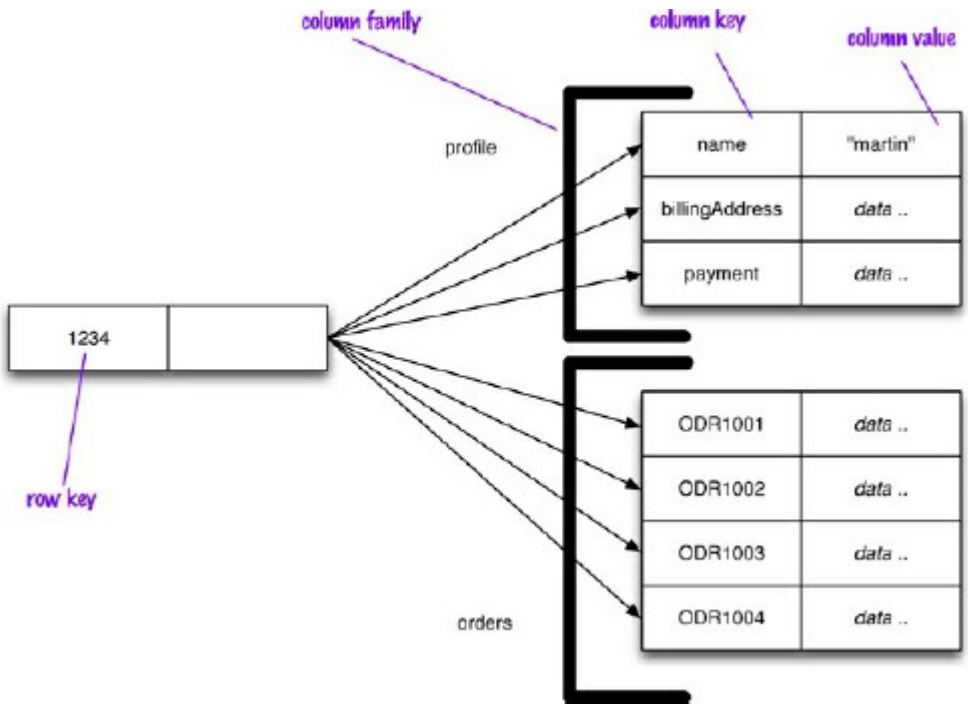
row-store

+ easy to add/modify a record
- might read in unnecessary data

column-store

+ only need to read in relevant data
- tuple writes require multiple accesses

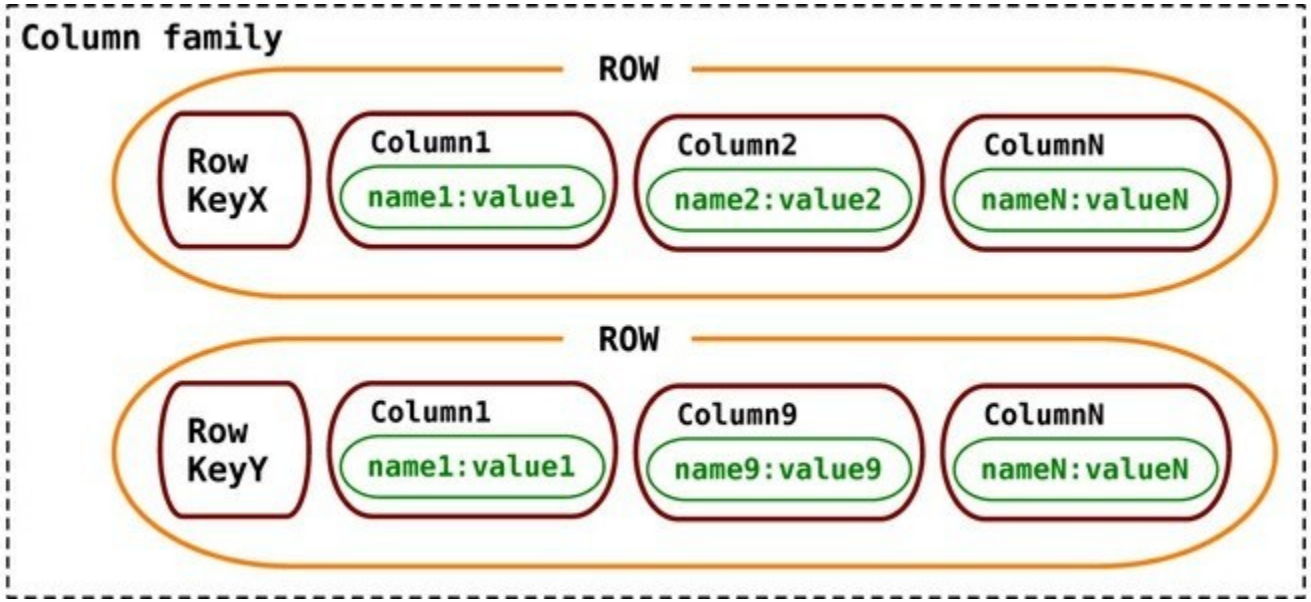
Column Family Database



Column Family Databases

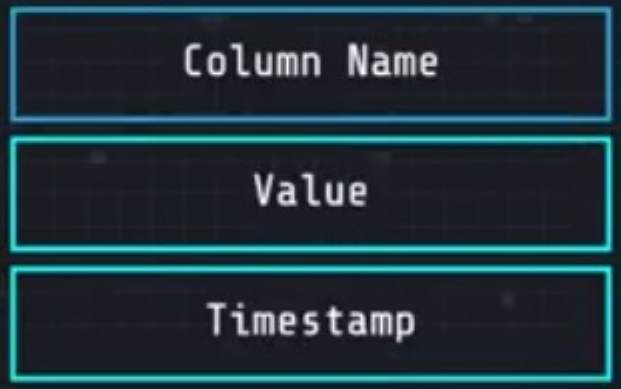
- Column-oriented databases are not good for queries that required to present the whole record of an entity. In this case, the row-store is better.
- Column-oriented storage is especially efficient when data readings are massive, and writes to a few columns. This is because in a query, only the data of the columns that interest are obtained, not all the columns of a record, which increases efficiency.
- The problem of writings would be when you want to write in all the columns of a record, since the columns work as individual units and are not necessarily contiguous as in the storage by rows.

Cassandra Datamodel with Column Families



Cassandra Model

- In Cassandra, the basic storage unit is a column, although there are super columns, families of columns and the keyspace. A column is a tuple that contains the name of the column, a value, and a timestamp.



Timestamps store the last update time of the column and are used for conflict resolution. A column name is analogous to an attribute name in a table in a relational database.

Example of a column

```
{
name: "fullName",
value: "Martin Fowler",
 timestamp: 12345667890
}
```

- The column has a key of firstName and the value of Martin and has a timestamp attached to it.
- A row is a collection of columns attached or linked to a key;
- a collection of similar rows makes a column family.
- When the columns in a column family are simple columns, the column family is known as **standard column family**.

Standard Column Family

//column family

```
{
//row
"pramod-sadalage" : {
firstName: "Pramod",
lastName: "Sadalage",
lastVisit: "2012/12/12"
}
```

```
//row
"martin-fowler" : {
firstName: "Martin",
lastName: "Fowler",
location: "Boston"
}
}
```

- Each column family can be compared to a container of rows in an RDBMS table where the key identifies the row and the row consists on multiple columns.
- The difference is that various rows do not have to have the same columns, and columns can be added to any row at any time without having to add it to other rows.
- We have the pramod-sadalage row and the martin-fowler row with different columns; both rows are part of the column family.

Super Column

- When a column consists of a map of columns, then we have a **super column**. A super column consists of a name and a value which is a map of columns. Think of a super column as a container of columns.

Super column

Example of Super Column

```
{
name: "book:978-0767905923",
value: {
author: "Mitch Albon",
title: "Tuesdays with Morrie",
isbn: "978-0767905923"
}
}
```

Column family

Super Column Family (When we use super columns to create a column family, we get a **super column family**.)

```
//super column family
{
//row
name: "billing:martin-fowler", value: {
address: {
name: "address:default",
value: {
 fullName: "Martin Fowler", street:"100 N. Main Street", zip: "20145"
}
},
billing: {
name: "billing:default",
value: {
 creditcard: "8888-8888-8888-8888", expDate: "12/2016"
}
}
//row
name: "billing:pramod-sadalage", value: {
address: {
name: "address:default",
value: {
 fullName: "Pramod Sadalage", street:"100 E. State Parkway", zip: "54130"
                                        }
},
billing: {
 name: "billing:default",
 value: {
  creditcard: "9999-8888-7777-4444", expDate: "01/2016"
 }
}
}
```

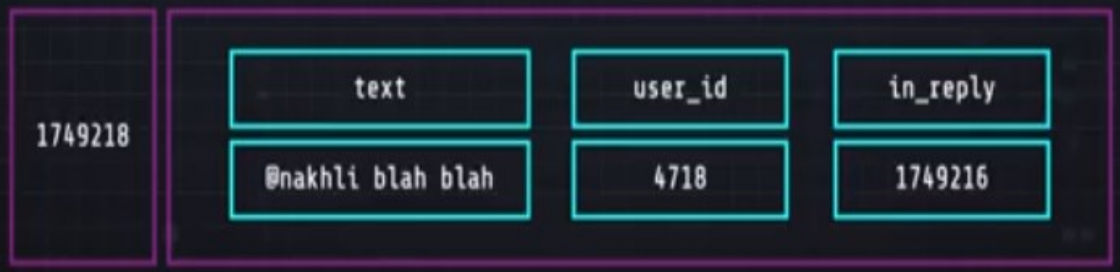
Super Column Family

- Super column families are good to keep related data together, but when some of the columns are not needed most of the time, the columns are still fetched and deserialized by Cassandra, which may not be optimal.
- Cassandra puts the standard and super column families into **keyspaces**. A keyspace is similar to a database in RDBMS where all column families related to the application are stored. Keyspaces have to be created so that column families can be assigned to them:

Keyspaces

Family of supercolumns "User Status"

Family of columns "Posts on Twitter: tweets":



Data Model

- **Column:** composed of a name, value and timestamp.

Cluster: are the machines that make up an instance of Cassandra. They can contain several Keyspaces.

Keyspace: namespace for a set of ColumFamily, associated with an application. Usually linked to a DB in the relational model.

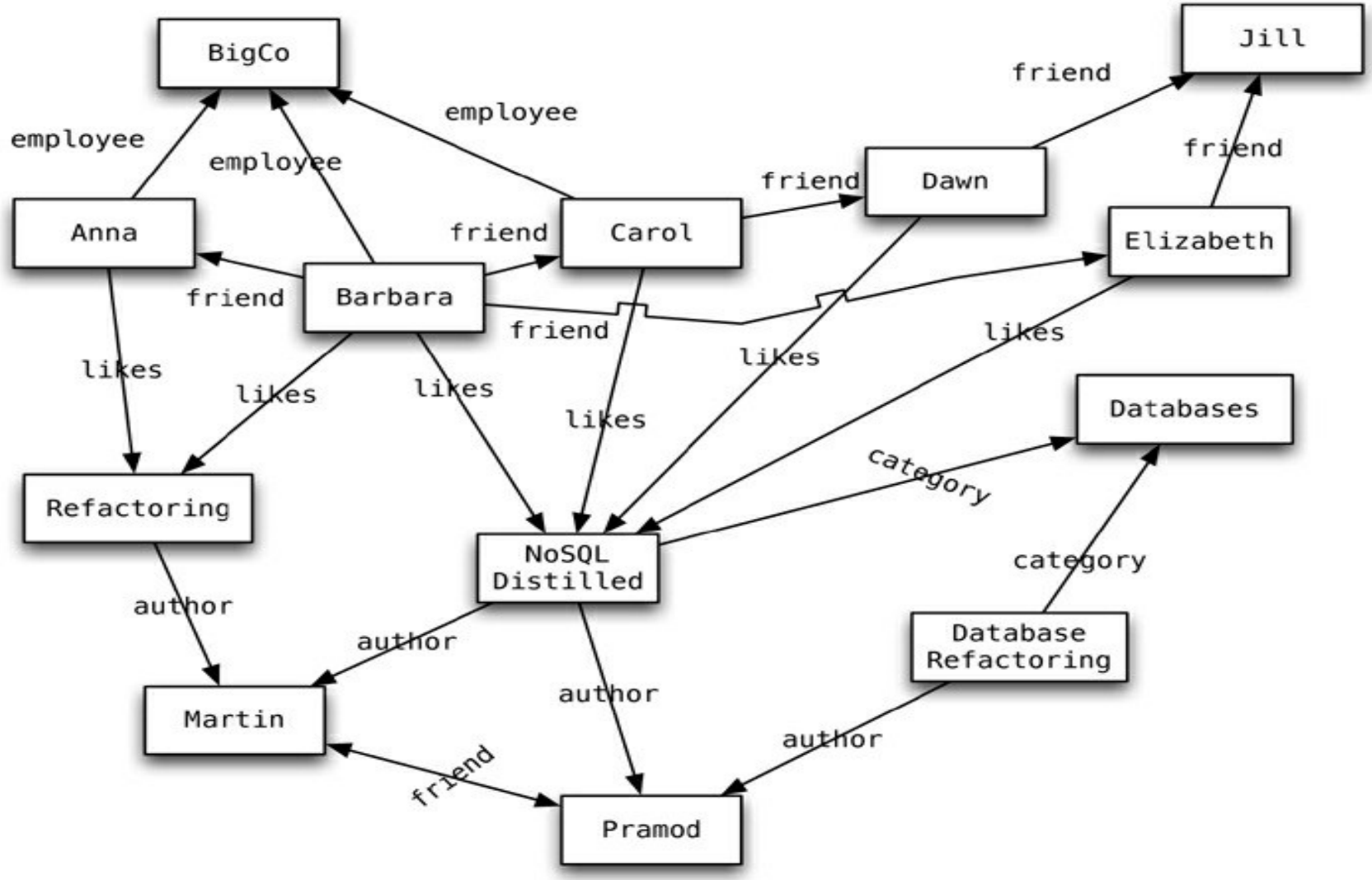
ColumFamily: contain multiple columns. They are usually linked. to a table in the relational model.

SuperColumn: columns that themselves have sub-columns.

Graph Databases

- In graph-oriented databases, graph is represented as a set of nodes or entities interconnected by edge or relationships.
- Graphs give importance not only to the data, but to the relations between them too.
- Relationships can also have attributes and direct queries can be made to relationships, rather than to the nodes.
- Being stored in this way, it is much more efficient to navigate between relationships than in a relational model.
- Obviously, these type of databases are only useful when the information can be easily represented as a network.
- Among the most used implementations are Neo4J, Hyperbase-DB, and InfoGrid.
- **The performance and scalability of a graph-oriented database are variable, highly flexible, and highly complex to implement.**

Graph Database



When to use - which

Graph-oriented database

	Performance	Scalability	Flexibility	Complexity	Functionality
Key-Value Stores	high	high	high	none	variable (none)
Column stores	high	high	moderate	low	minimal
Document stores	high	variable (high)	high	low	variable (low)
Graph databases	variable	variable	high	high	graph theory
Relational databases	variable	variable	low	moderate	relational algebra

Schema less Databases

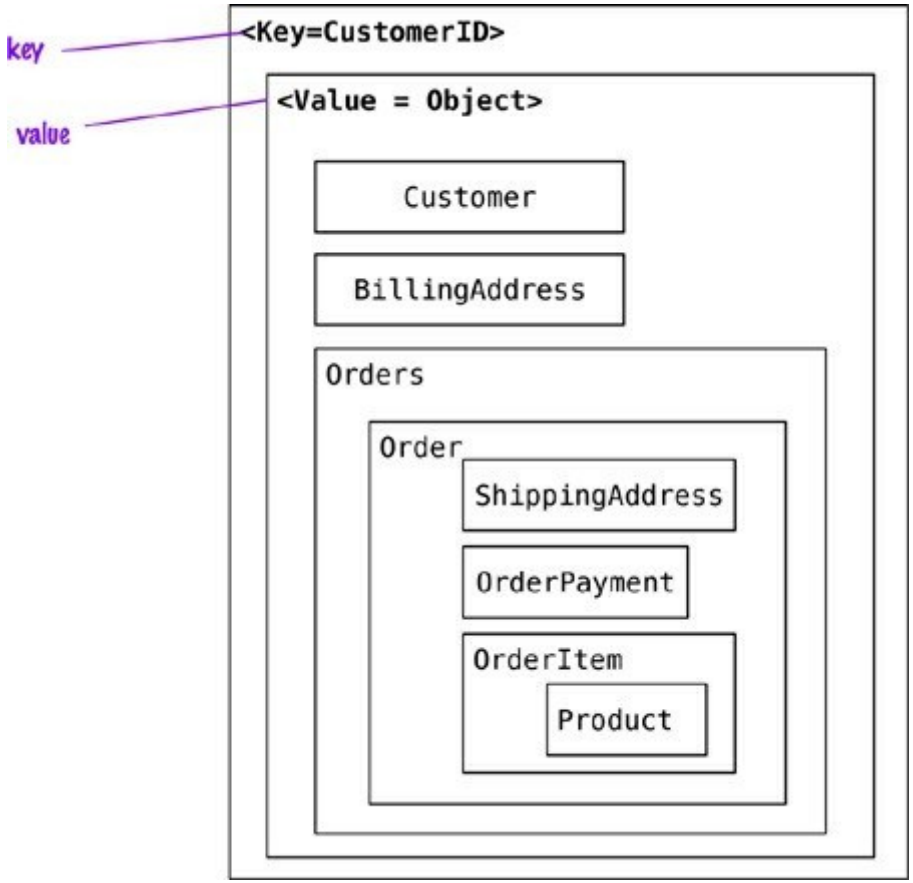
- RDBMS has restriction on kind of data which needs to be stored based on data types of columns.
- NoSQL has freedom. Any document or value can has its own fields.
- Challenge move to applications.
- Schema moves into application.

Materialized Views

- Aggregates – Advantage/Disadvantage
- Views in RDBMS
- Materialised View
  - Eager View
  - Computational View
  - Summary View

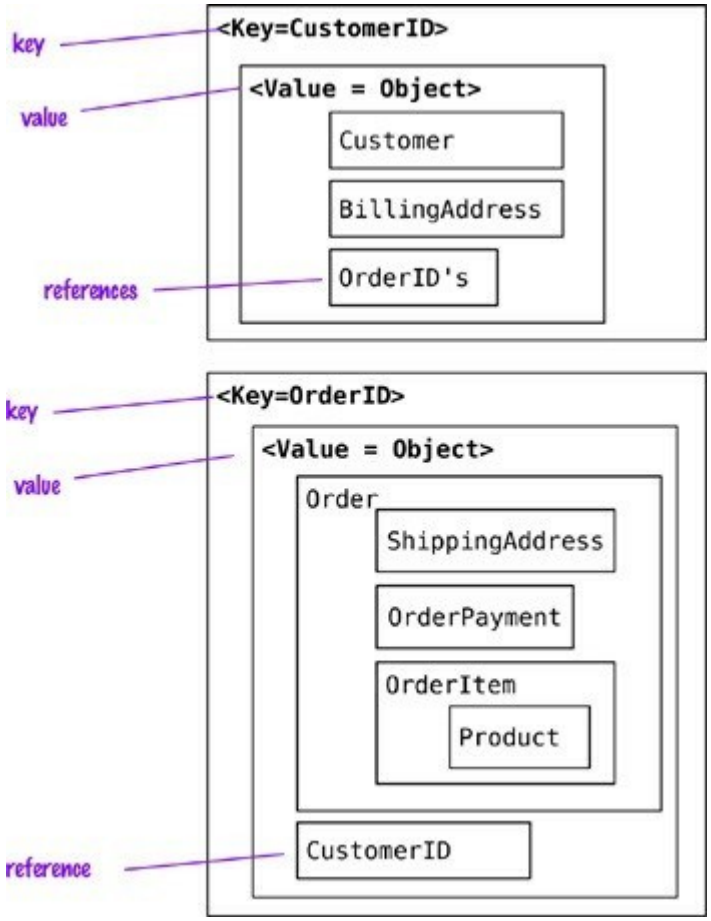
Modelling for Data Access

- Key-Value store



Document store with references

- Reference



Document Database with reference

```
# Customer object
{
"customerId": 1, "customer": {
"name": "Martin",
"billingAddress": [{"city": "Chicago"}],
"payment": [{"type": "debit","ccinfo": "1000-1000-1000-1000"}],
"orders":[{"orderId":99}]
}
}
```

```
# Order object
{
"customerId": 1, "orderId": 99, "order":{
"orderDate":"Nov-20-2011", "orderItems":[{"productId":27,
 "price": 32.45}],
 "orderPayment":[{"ccinfo":"1000-1000-1000-1000",
    "txnId":"abelif879rft"}],
"shippingAddress":{"city":"Chicago"}
}
}
```

Business Analytics with Aggregates

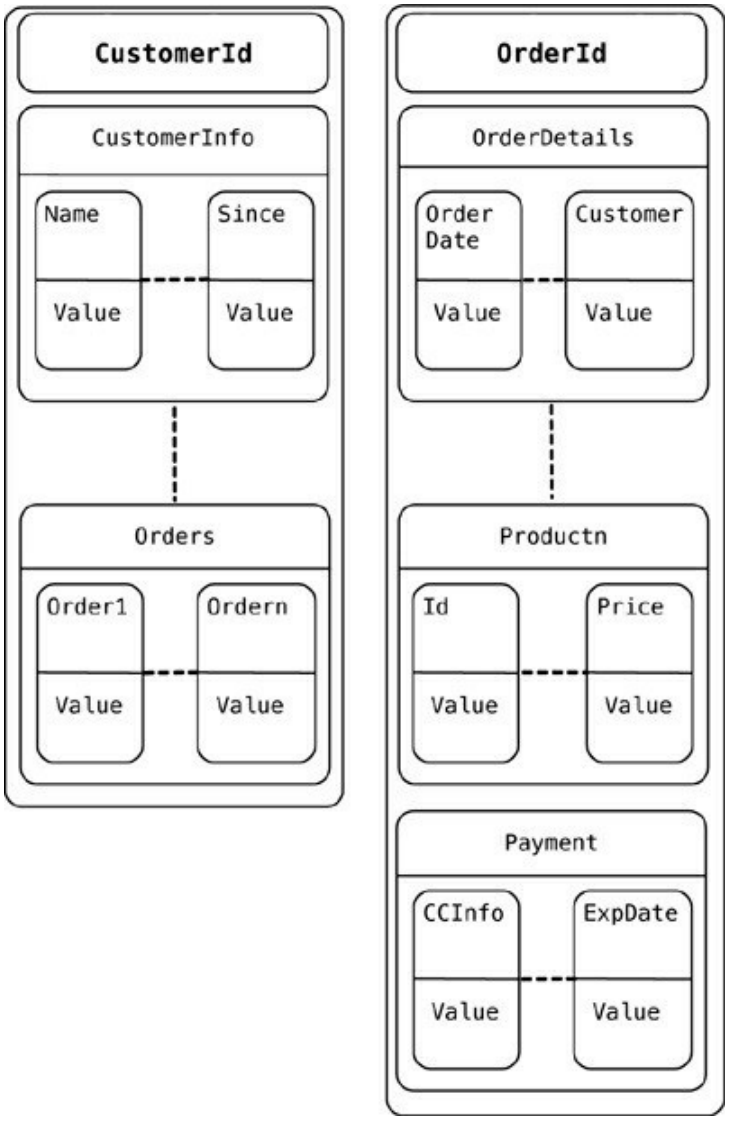
- Materialised View (which order has given product in them)

```
{ "itemid":27,
"orders":
{99545,897,678}
}
{ "itemid":29,
"orders":
{199,545,704,819}
}
```

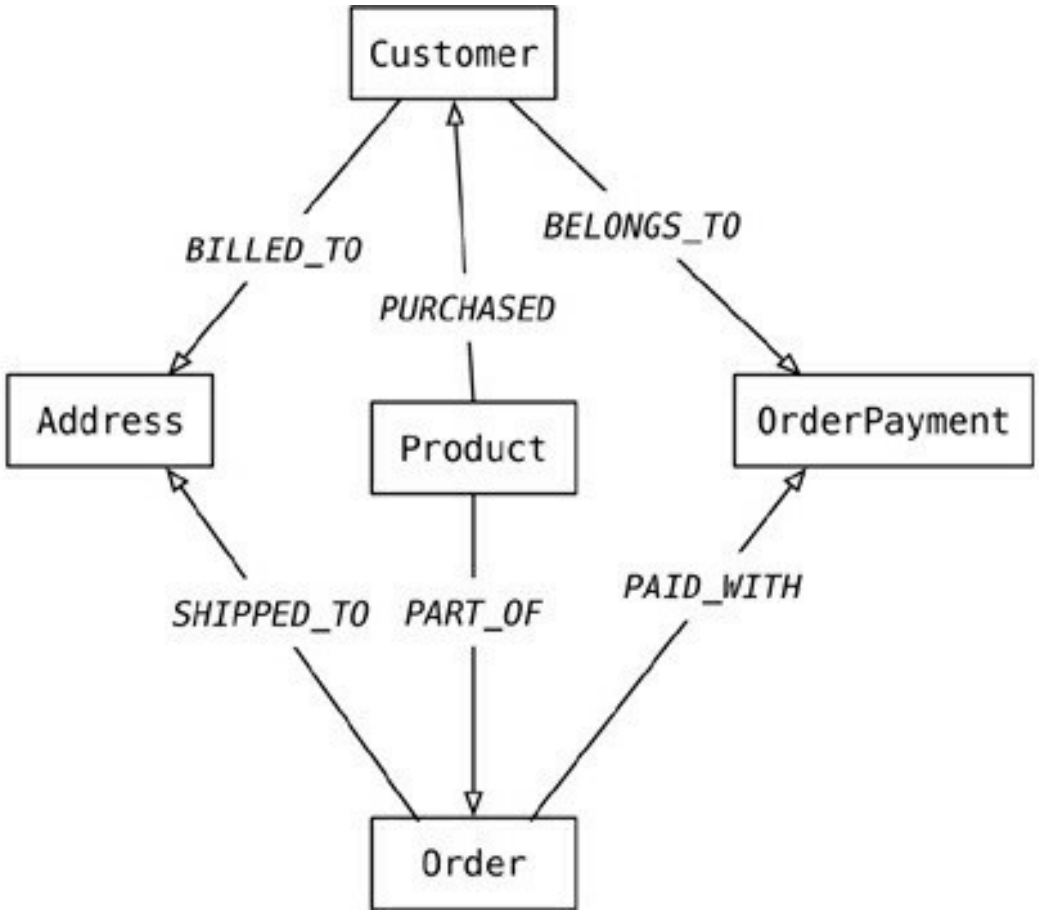
Removing References from Document Storage

```
# Customer object
{
"customerId": 1, "name": "Martin",
"billingAddress": [{"city": "Chicago"}], "payment": [
{"type": "debit",
"ccinfo": "1000-1000-1000-1000"} //No need to update customer record with order id.
]
}
# Order object
{
"orderId": 99, "customerId": 1, "orderDate":"Nov-20-2011",
"orderItems":[{"productId":27, "price": 32.45}], "orderPayment":[{"ccinfo":"1000-1000-1000-
   1000",
"txnId":"abelif879rft"}],
"shippingAddress":{"city":"Chicago"}
}
```

Column Family Database



Graph Database



Distribution Models

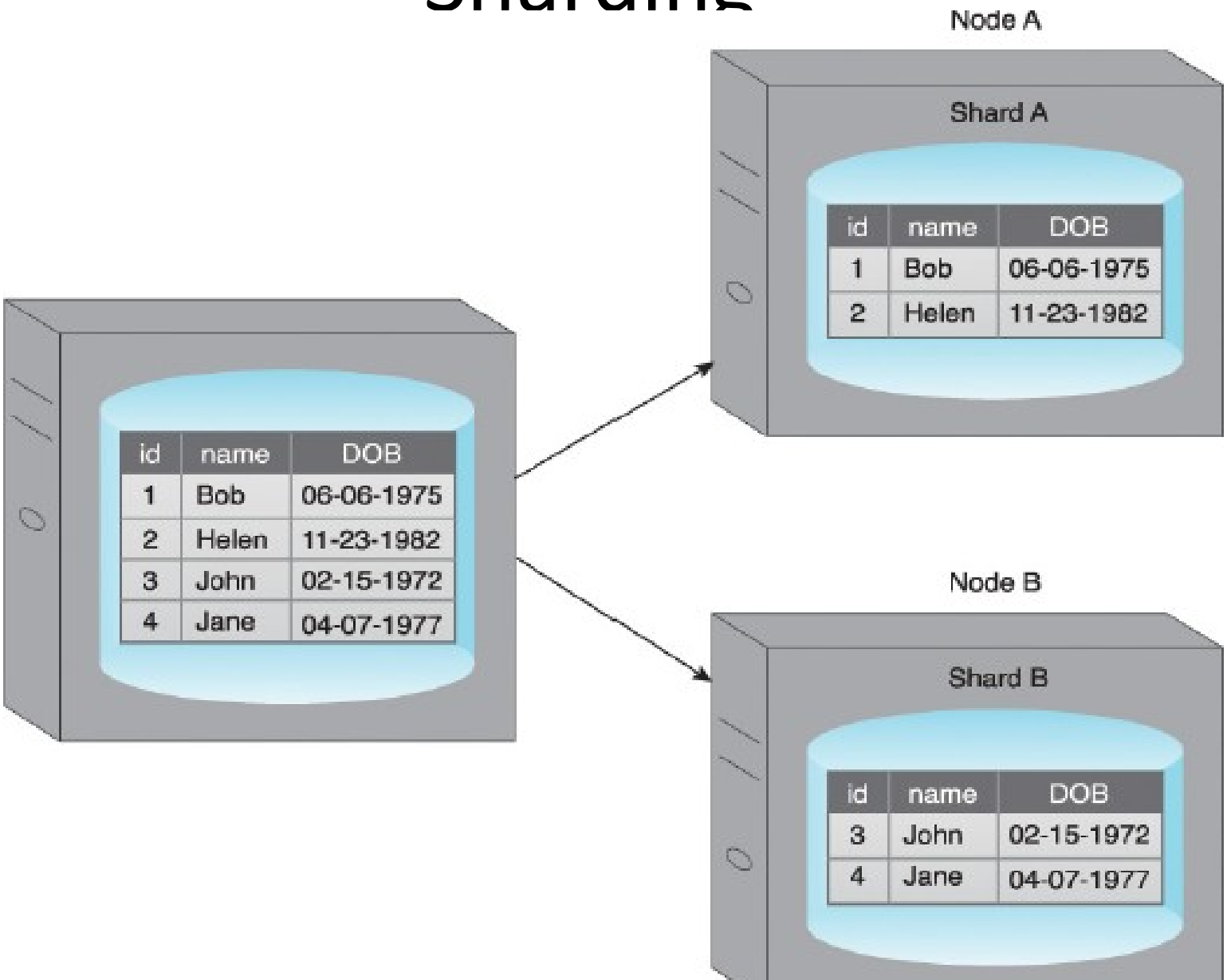
Background

- NoSQL primarily for running on Clusters
- Can scale better
- Mainly two paths for data distribution
  - Sharding
  - Replication
    - Master-Slave
    - Peer-to-Peer

Sharding

- Horizontally partitioning large datasets into a collection of smaller more manageable datasets called shards.
- Shards are distributed across multiple nodes (machine), sharing the same schema and collectively represent complete dataset.
- Scalability is achieved by distributing the processing load across various nodes and it could be easily enhanced by adding more powerful resources to existing infrastructure.
- As each node is only responsible for their data, the read/writes are greatly improved.

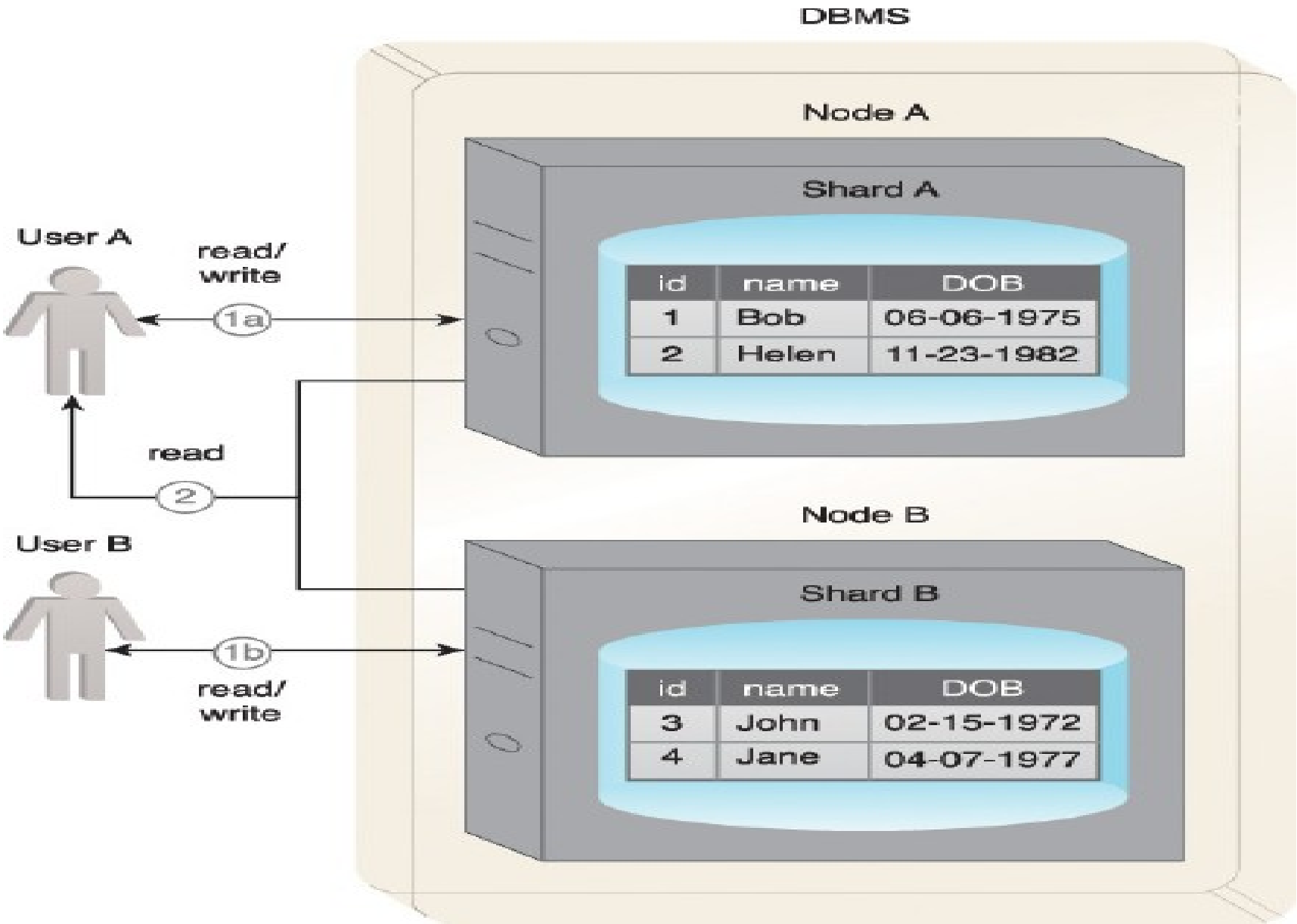
Sharding


Node A
Shard A
id name DOB
1 Bob 06-06-1975
2 Helen 11-23-1982
id name DOB
1 Bob 06-06-1975
2 Helen 11-23-1982
3 John 02-15-1972
4 Jane 04-07-1977
Node B
Shard B
id name DOB
3 John 02-15-1972
4 Jane 04-07-1977

Sharding

- Each shard can independently service reads and writes for the specific subset of data that it is responsible for.
- Shards provide some resilience against failures as only the data of failed node will be inaccessible.
- Query pattern needs to be taken care.
- Depending upon the query the data may need to be fetched from multiple the shards.
- Query requiring data from multiple shards may impact performance.
- Data locality keeps commonly accessed data co-located on a single shard and improve performance.

Sharding


DBMS
Node A
Shard A
id name DOB
1 Bob 06-06-1975
2 Helen 11-23-1982
User A
read/
write
1a
read
2
User B
1b
read/
write
Node B
Shard B
id name DOB
3 John 02-15-1972
4 Jane 04-07-1977

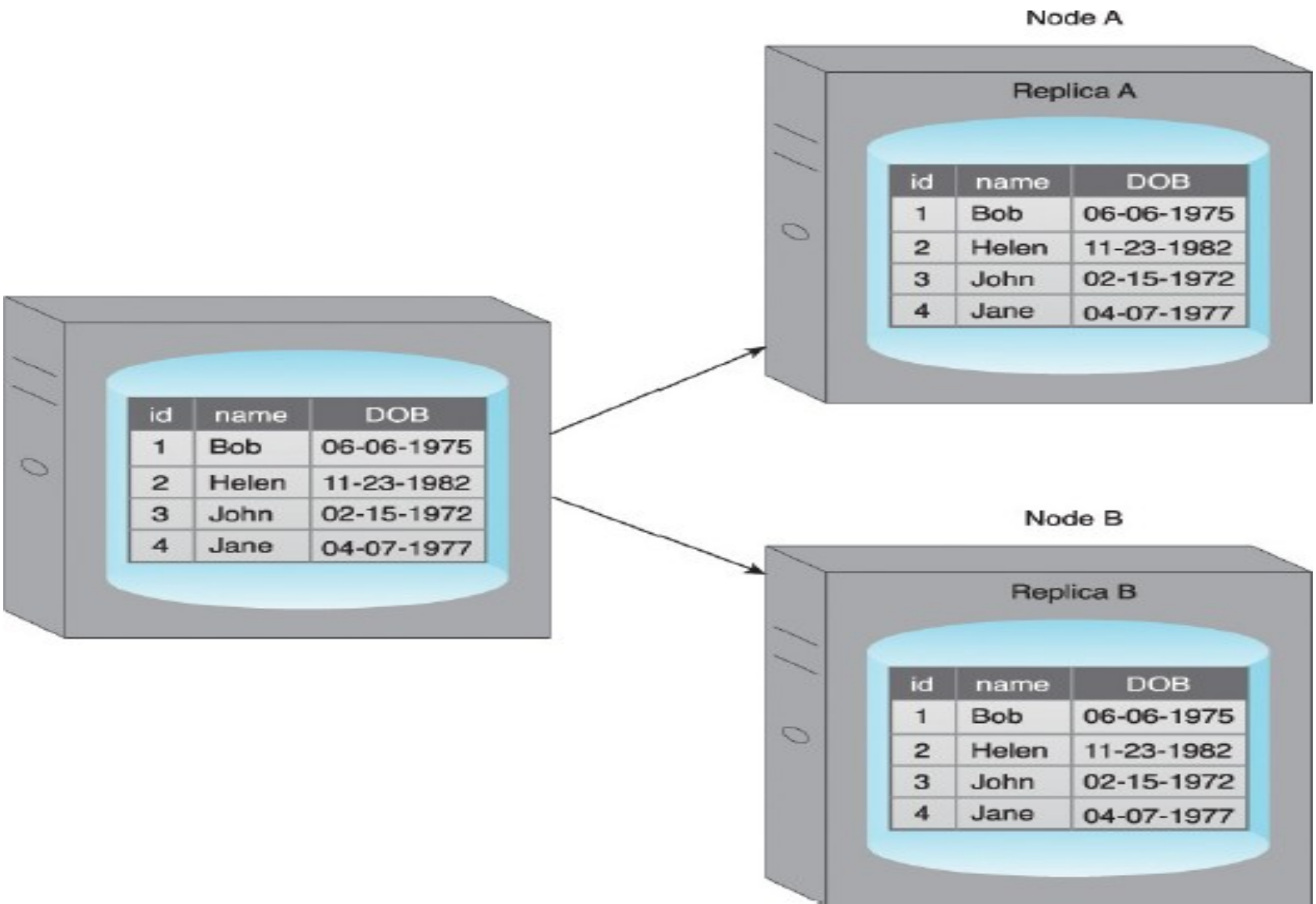
Replication

- Replication stores multiple copies of the datasets, known as, replicas on multiple nodes.
- Scalability and Availability is the result.
- Fault tolerance

Two types of replication strategies

- Master-Slave
- Peer-to-Peer

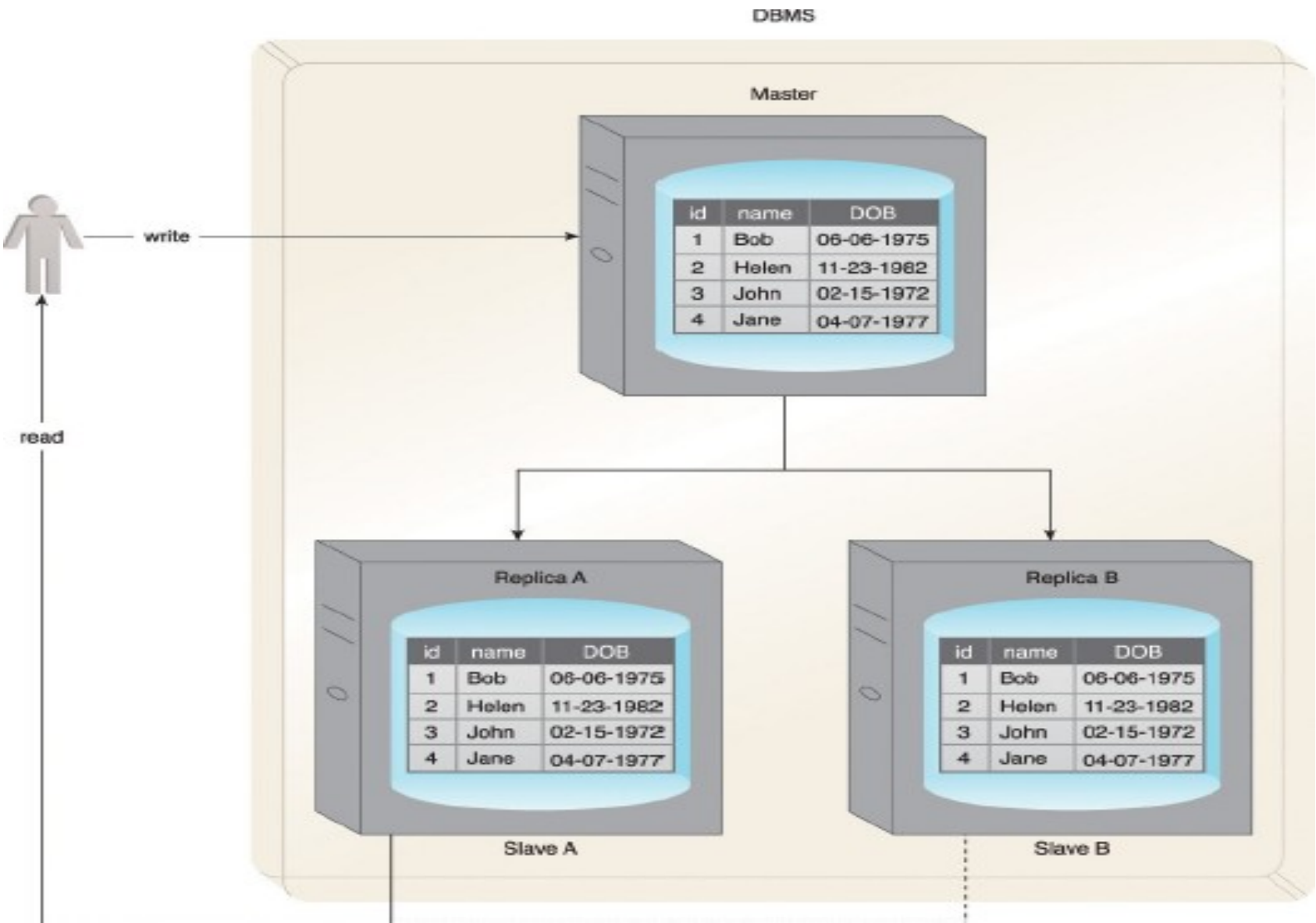
Replication



Master-Slave

- During master-slave replication, nodes are arranged in a master-slave configuration, and all data is written to a master node. Once saved, the data is replicated over to multiple slave nodes.
- All external write requests, including insert, update and delete, occur on the master node, whereas read requests can be fulfilled by any slave node.
- Master-slave replication is ideal for read intensive loads rather than write intensive loads since growing read demands can be managed by horizontal scaling to add more slave nodes.
- Writes are consistent, as all writes are coordinated by the master node.
- The implication is that write performance will suffer as the amount of writes increases. If the master node fails, reads are still possible via any of the slave nodes.

Master-Slave - Writes are managed by the master node and data can be read from either Slave A or Slave B.



Master-Slave

- A slave node can be configured as a backup node for the master node. In the event that the master node fails, writes are not supported until a master node is re-established. The master node is either resurrected from a backup of the master node, or a new master node is chosen from the slave nodes.

- One concern with master-slave replication is read inconsistency, which can be an issue if a slave node is read prior to an update to the master being copied to it.

A scenario where read inconsistency occurs.

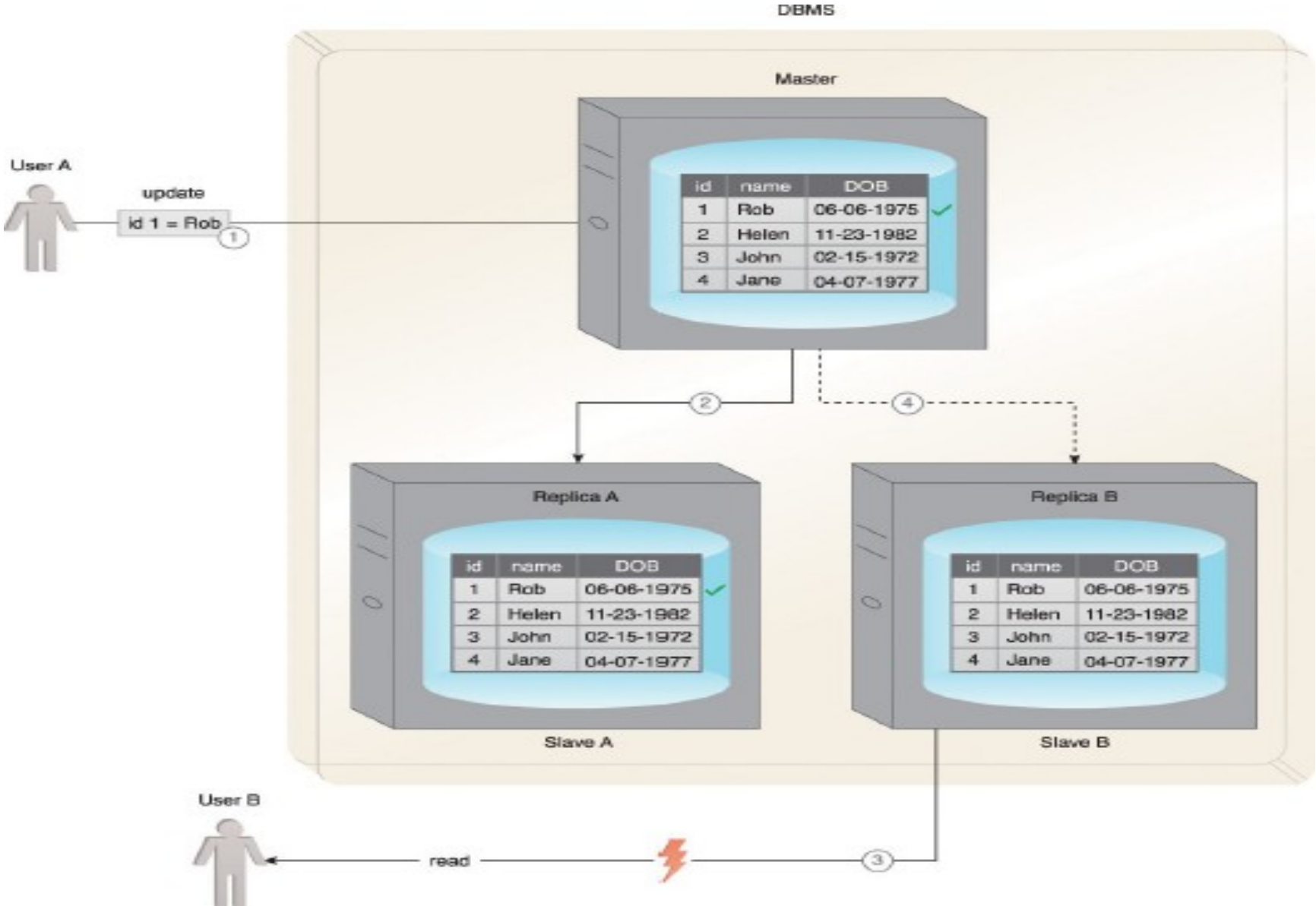
1. User A updates data.

2. The data is copied over to Slave A by the Master.

3. Before the data is copied over to Slave B, User B tries to read the data from Slave B, which results in an inconsistent read.

4. The data will eventually become consistent when Slave B is updated by the Master.

Read Inconsistency



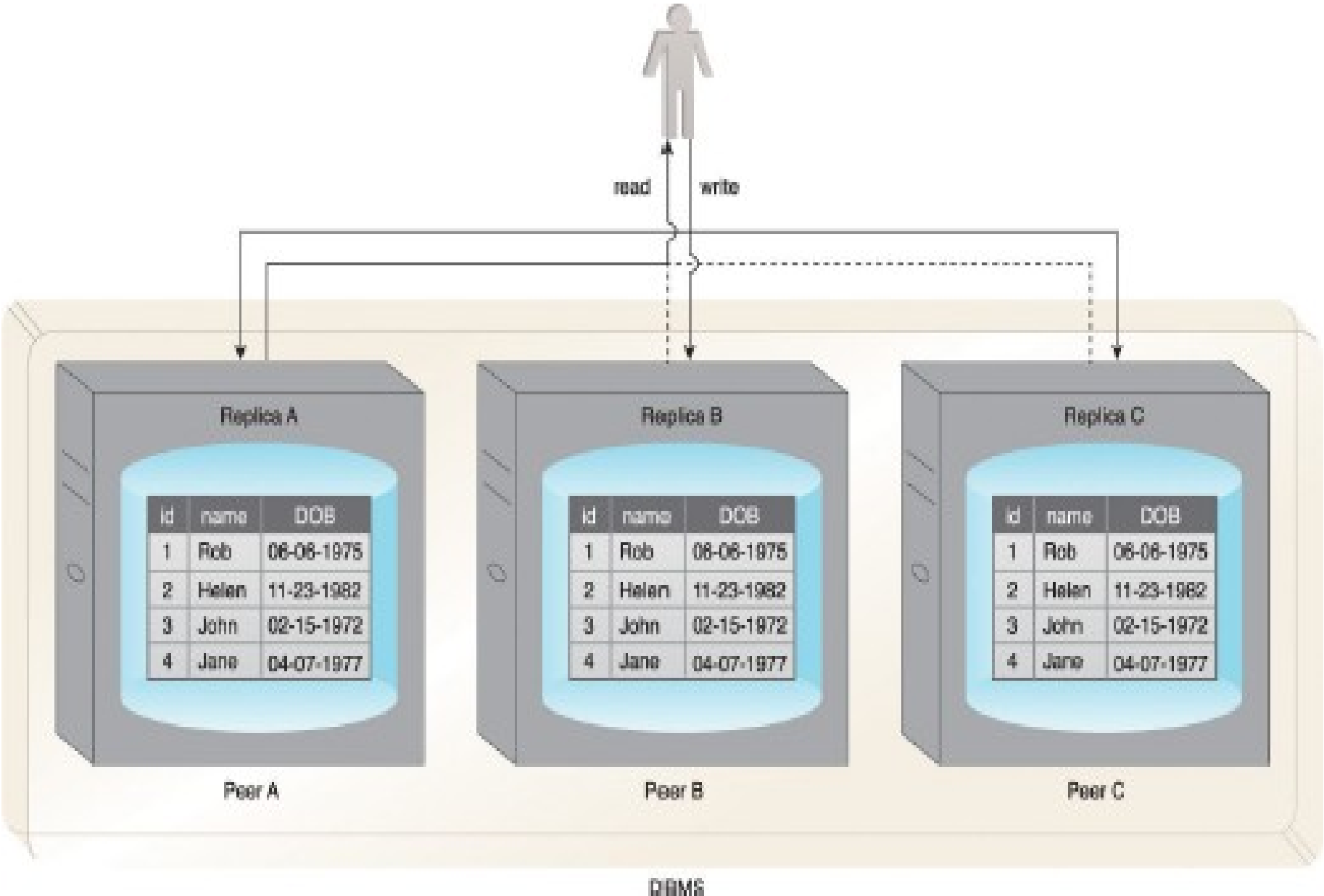
Read Inconsistency

- To ensure read consistency, a voting system can be implemented where a read is declared consistent if the majority of the slaves contain the same version of the record. Implementation of such a voting system requires a reliable and fast communication mechanism between the slaves.

Peer-to-Peer

- With peer-to-peer replication, all nodes operate at the same level. In other words, there is not a master-slave relationship between the nodes. Each node, known as a peer, is equally capable of handling reads and writes. Each write is copied to all peers.

Peer-to-Peer


read
write
Replica A
id name DOB
1 Rob 06-06-1975
2 Helen 11-23-1982
3 John 02-15-1972
4 Jane 04-07-1977
Peer A
Replica B
id name DOB
1 Rob 06-06-1975
2 Helen 11-23-1982
3 John 02-15-1972
4 Jane 04-07-1977
Peer B
Replica C
id name DOB
1 Rob 06-06-1975
2 Helen 11-23-1982
3 John 02-15-1972
4 Jane 04-07-1977
Peer C
DBMS

Write-Inconsistency

- Peer-to-peer replication is prone to write inconsistencies that occur as a result of a simultaneous update of the same data across multiple peers. This can be addressed by implementing either a pessimistic or optimistic concurrency strategy.

- Pessimistic concurrency is a proactive strategy that prevents inconsistency. It uses locking to ensure that only one update to a record can occur at a time.

Write-Inconsistency

However, this is detrimental to availability since the database record being updated remains unavailable until all locks are released.

- Optimistic concurrency is a reactive strategy that does not use locking. Instead, it allows inconsistency to occur with knowledge that eventually consistency will be achieved after all updates have propagated. With optimistic concurrency, peers may remain inconsistent for some period of time before attaining consistency. However, the database remains available as no locking is involved.

Read-Inconsistency

- Like master-slave replication, reads can be inconsistent during the time period when some of the peers have completed their updates while others perform their updates.
- However, reads eventually become consistent when the updates have been executed on all peers.

Read-Inconsistency

- A scenario where an inconsistent read occurs.

1. User A updates data.

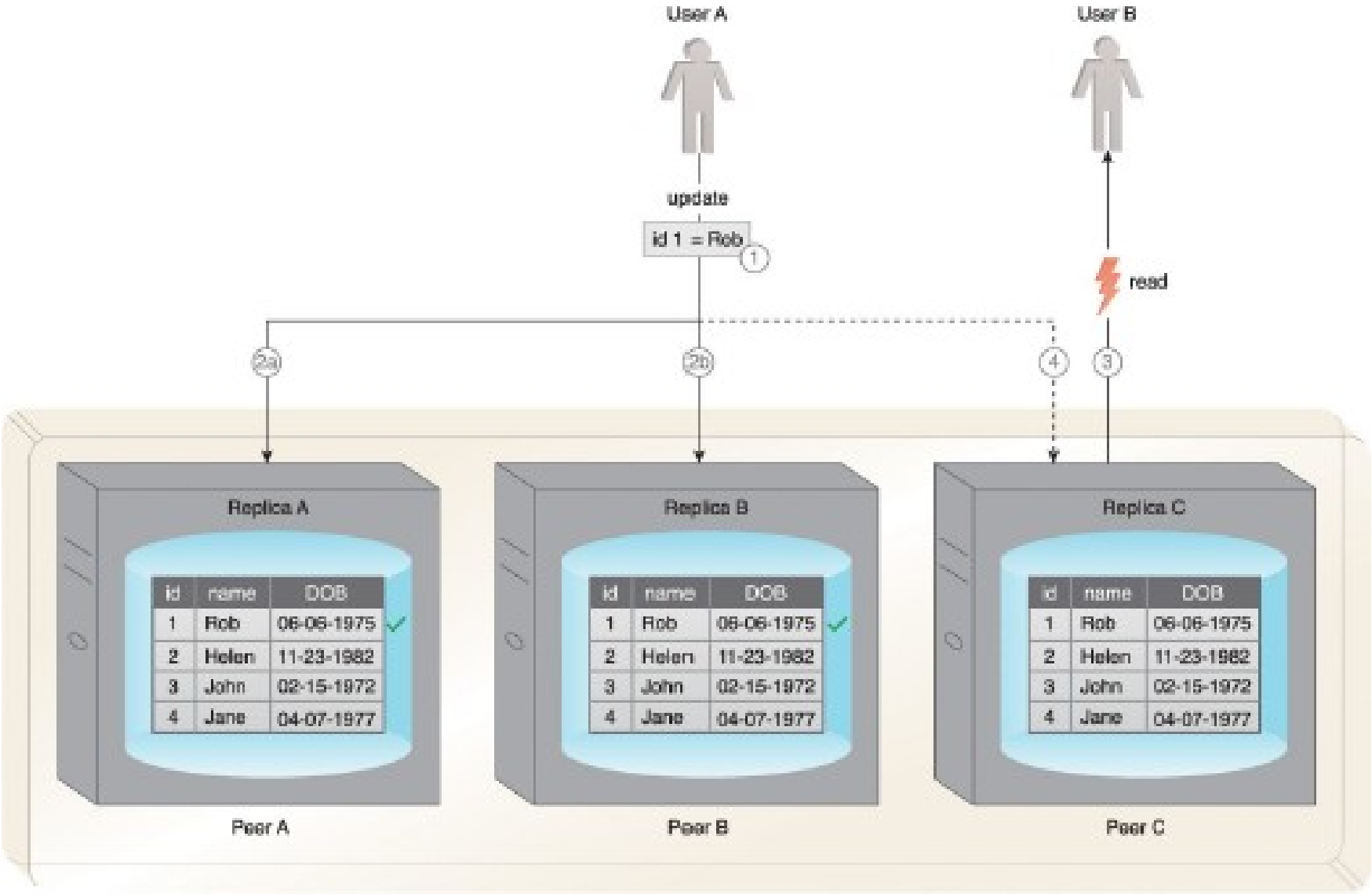
2. a. The data is copied over to Peer A.

- **b. The data is copied over to Peer B.**

3. Before the data is copied over to Peer C, User B tries to read the data from Peer C, *resulting in an inconsistent read.*

4. The data will eventually be updated on Peer C, and the database will once again *become consistent.*

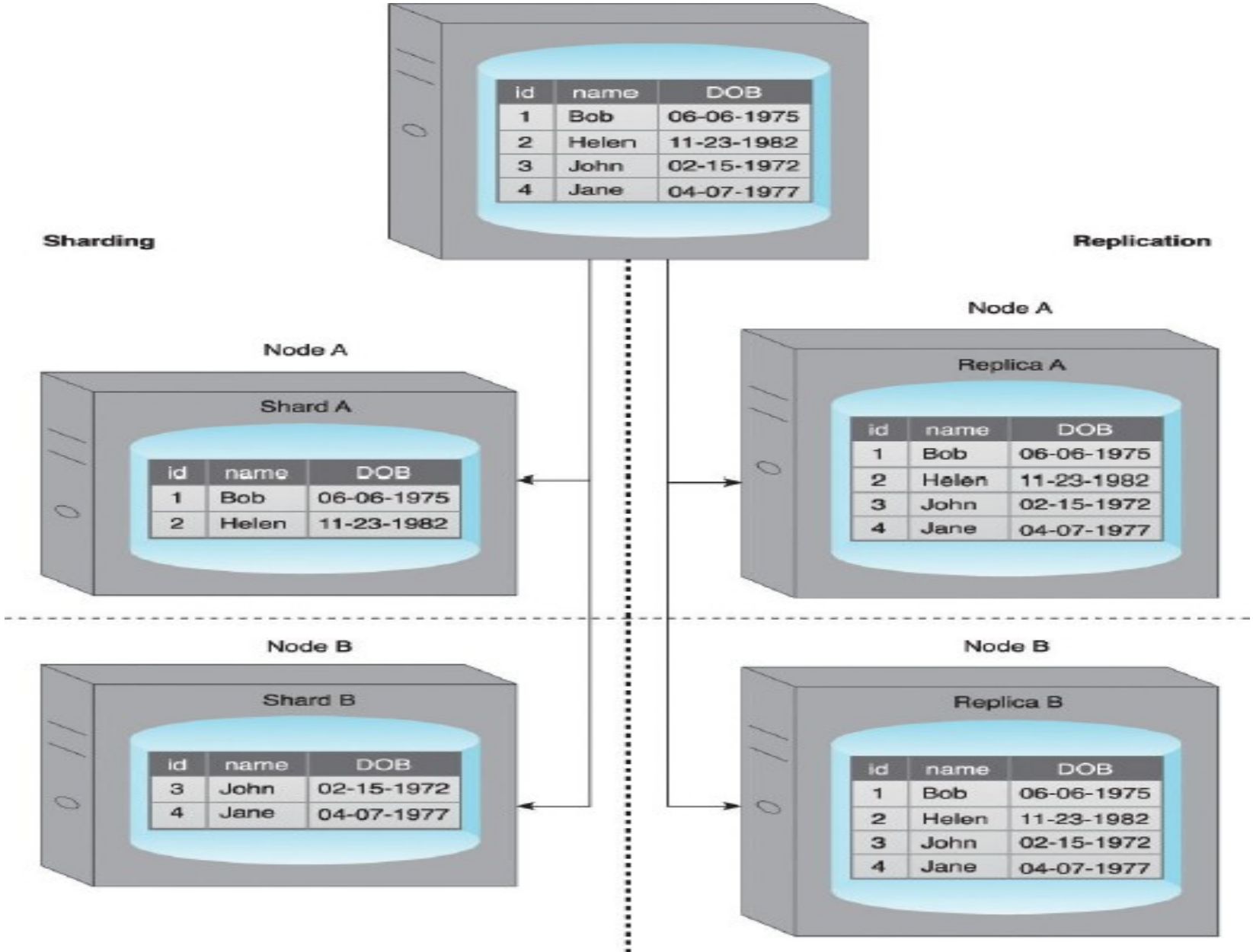
Read-Inconsistency



Sharding and Replication

- To improve on the limited fault tolerance offered by sharding, while additionally benefiting from the increased availability and scalability of replication, both sharding and replication can be combined
  - Sharding and Master-Slave replication
  - Sharding and Peer-to-Peer replication

Sharding and Replication



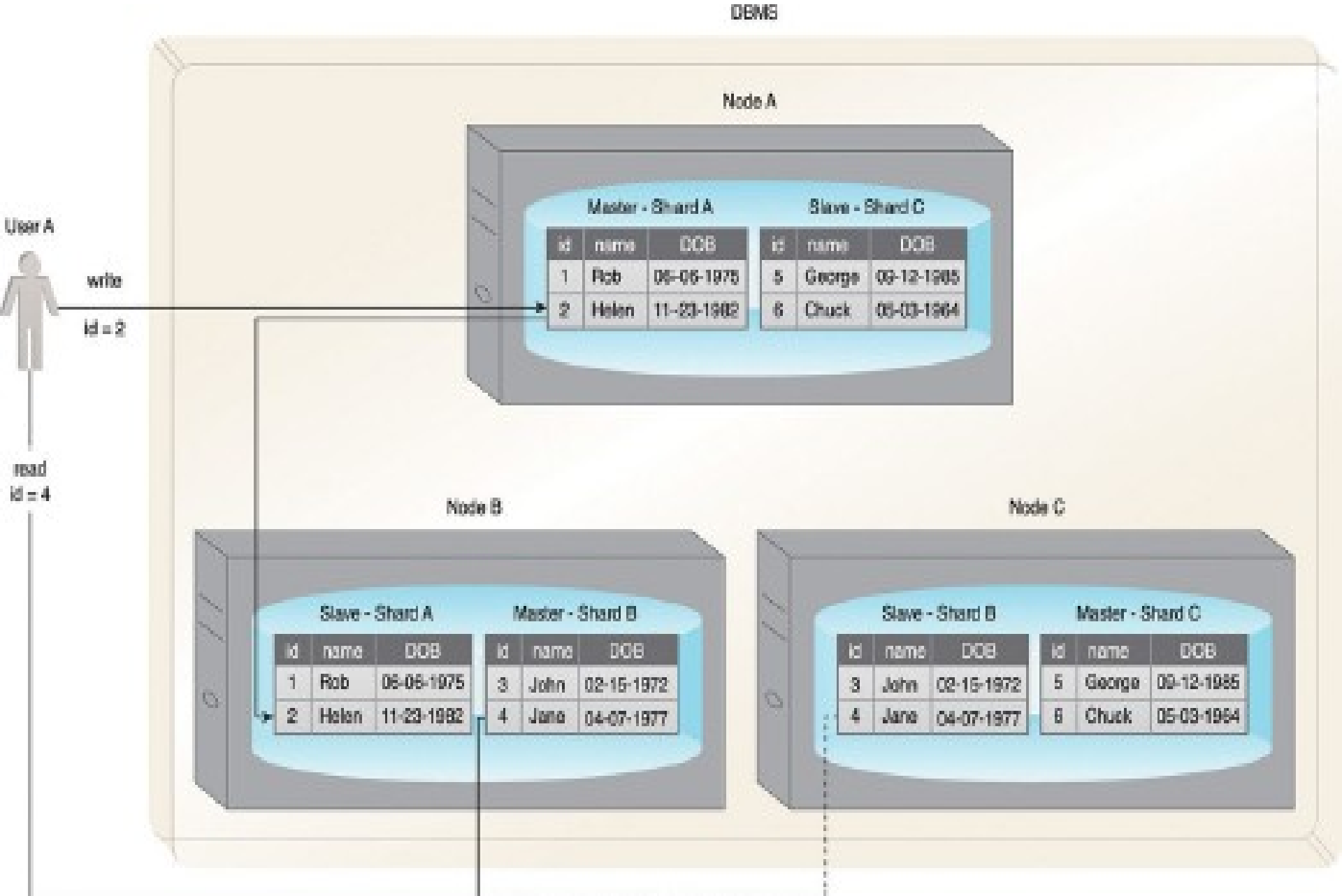
Sharding and Master-Slave

- When sharding is combined with master-slave replication, multiple shards become slaves of a single master, and the master itself is a shard.
- This results in multiple masters, but a single slave-shard can only be managed by a single master-shard.
- Write consistency is maintained by the master-shard. However, if the master-shard becomes non-operational or a network outage occurs, fault tolerance with regards to write operations is impacted.
- Replicas of shards are kept on multiple slave nodes to provide scalability and fault tolerance for read operations.

Sharding and Master-Slave

- Each node acts both as a master and a slave for different shards.
- Writes (id = 2) to Shard A are regulated by Node A, as it is the master for Shard A.
- Node A replicates data (id = 2) to Node B, which is a slave for Shard A.
- Reads (id = 4) can be served directly by either Node B or Node C as they each contain Shard B.

Sharding & Master-Slave



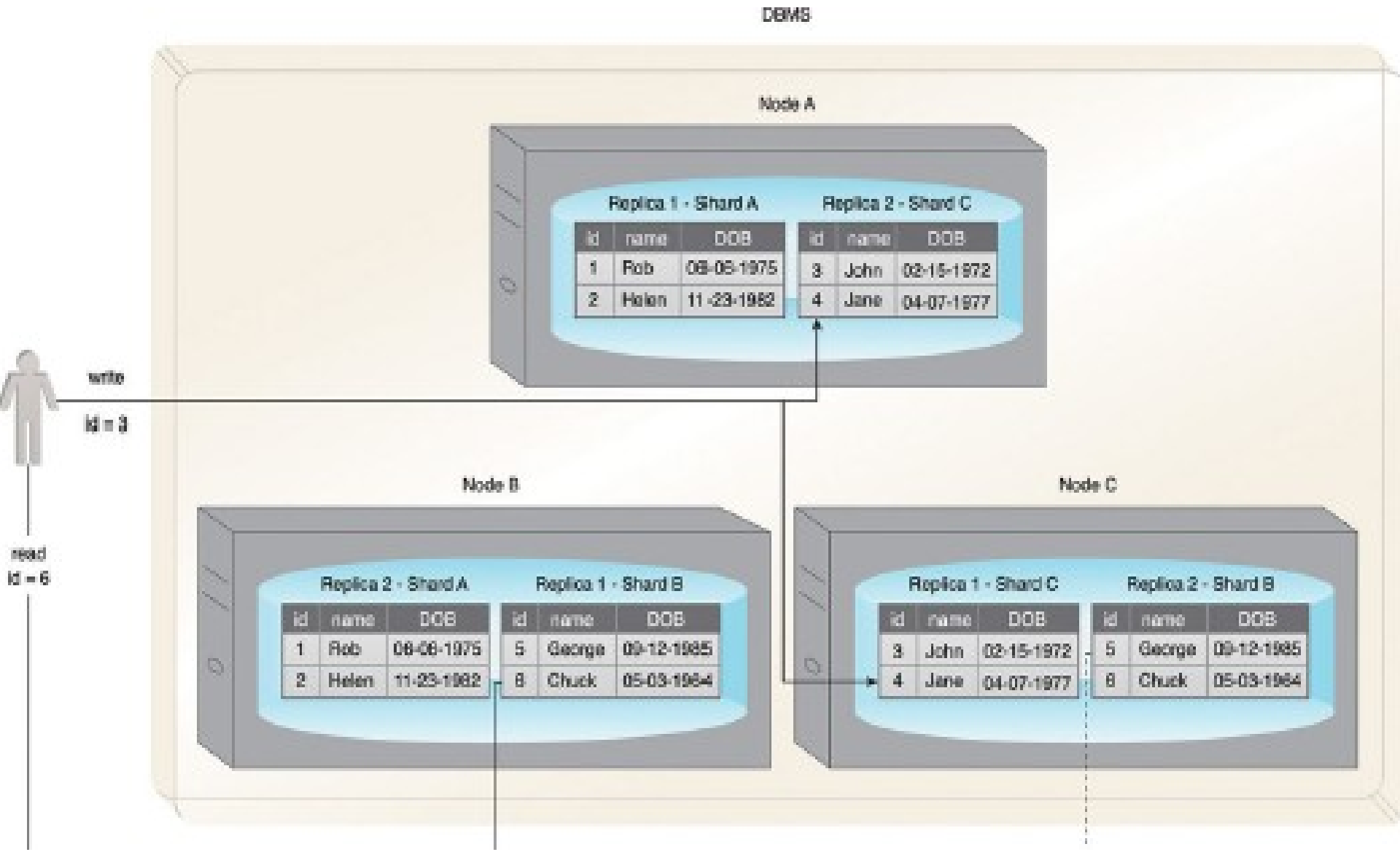
Sharding and Peer-to-Peer

- When combining sharding with peer-to-peer replication, each shard is replicated to multiple peers, and each peer is only responsible for a subset of the overall dataset.
- Collectively, this helps achieve increased scalability and fault tolerance. As there is no master involved, there is no single point of failure and fault-tolerance for both read and write operations is supported.

Sharding and Peer-to-Peer

- Each node contains replicas of two different shards.
- Writes (id = 3) are replicated to both Node A and Node C (Peers) as they are responsible for Shard C.
- Reads (id = 6) can be served by either Node B or Node C as they each contain Shard B.

Peer-to-Peer


DBMS
Node A
Replica 1 - Shard A
id name DOB
1 Rob 08-06-1975
2 Helen 11-23-1982
Replica 2 - Shard C
id name DOB
3 John 02-15-1972
4 Jane 04-07-1977
write
id = 3
read
id = 6
Node B
Replica 2 - Shard A
id name DOB
1 Rob 08-06-1975
2 Helen 11-23-1982
Replica 1 - Shard B
id name DOB
5 George 09-12-1985
8 Chuck 05-03-1964
Node C
Replica 1 - Shard C
id name DOB
3 John 02-15-1972
4 Jane 04-07-1977
Replica 2 - Shard B
id name DOB
5 George 09-12-1985
8 Chuck 05-03-1964

CAP Theorem

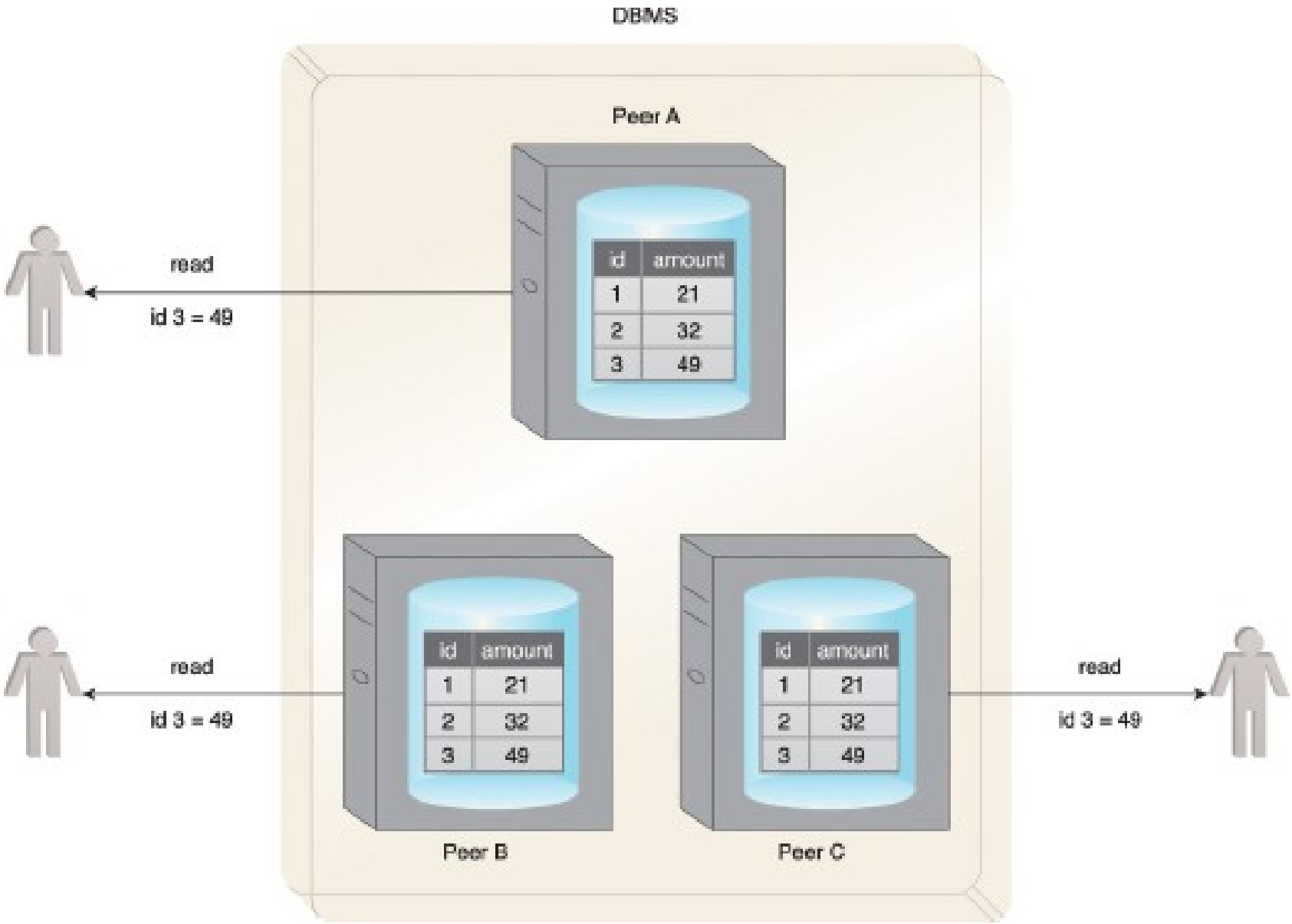
- The Consistency, Availability, and Partition tolerance (CAP) theorem, also known as Brewer's theorem, expresses a triple constraint related to distributed database systems.
- It states that a distributed database system, running on a cluster, can only provide two of the following three properties:

CAP Theorem

- *Consistency – A read from any node results in the same data across multiple nodes.*

- *E.g. In the following figure all three users get the same value for the amount column even though three different nodes are serving the record.*

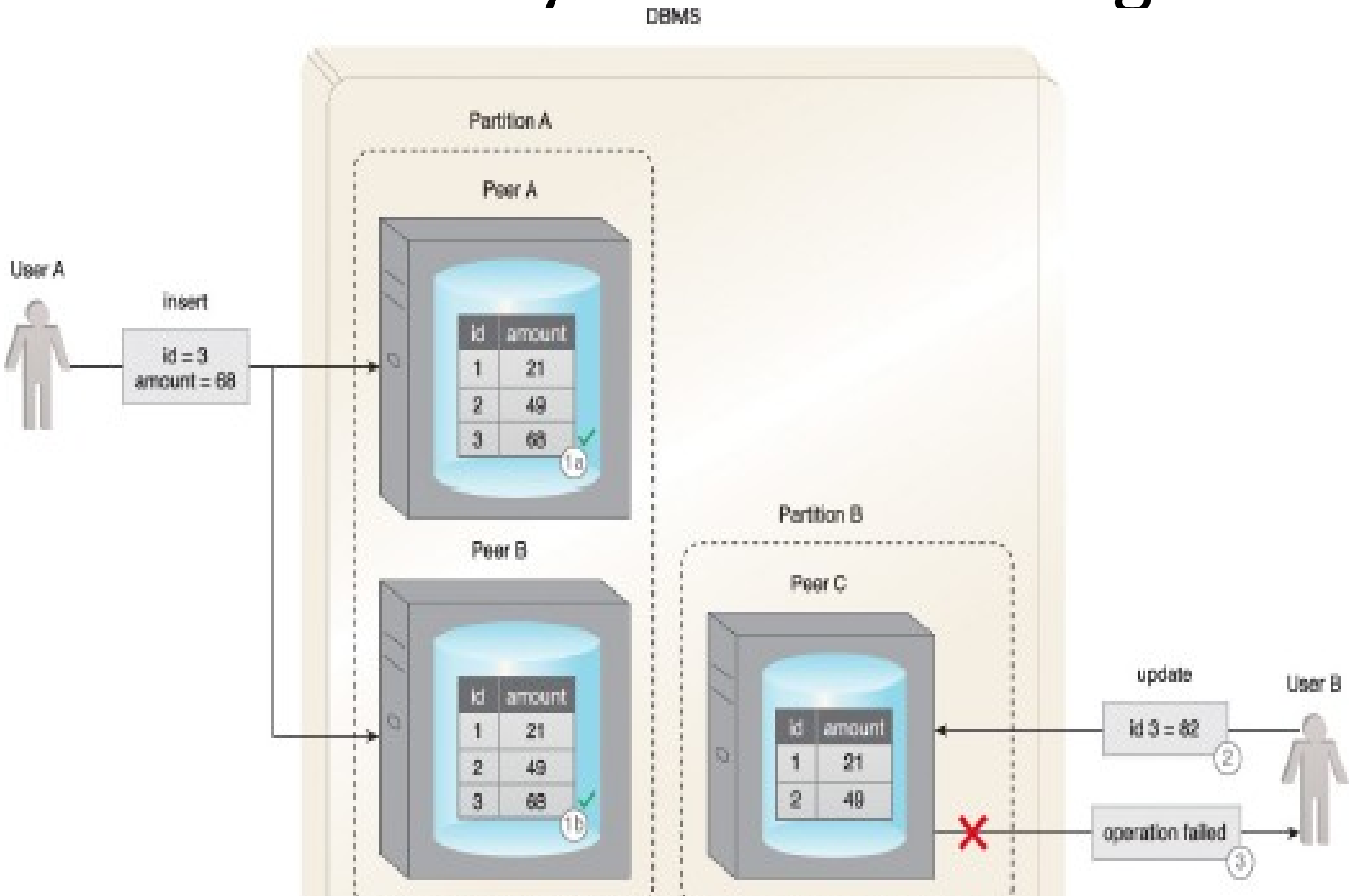
Consistency



Availability and Partitioning

- *Availability – A read/write request to a non-failing node will always be acknowledged in the form of a success or a failure. In other words request to a failed node doesn't mean lack of availability.*
- *Partition tolerance – The database system can tolerate communication outages that split the cluster into multiple silos and can still service read/write requests*

Availability and Partitioning

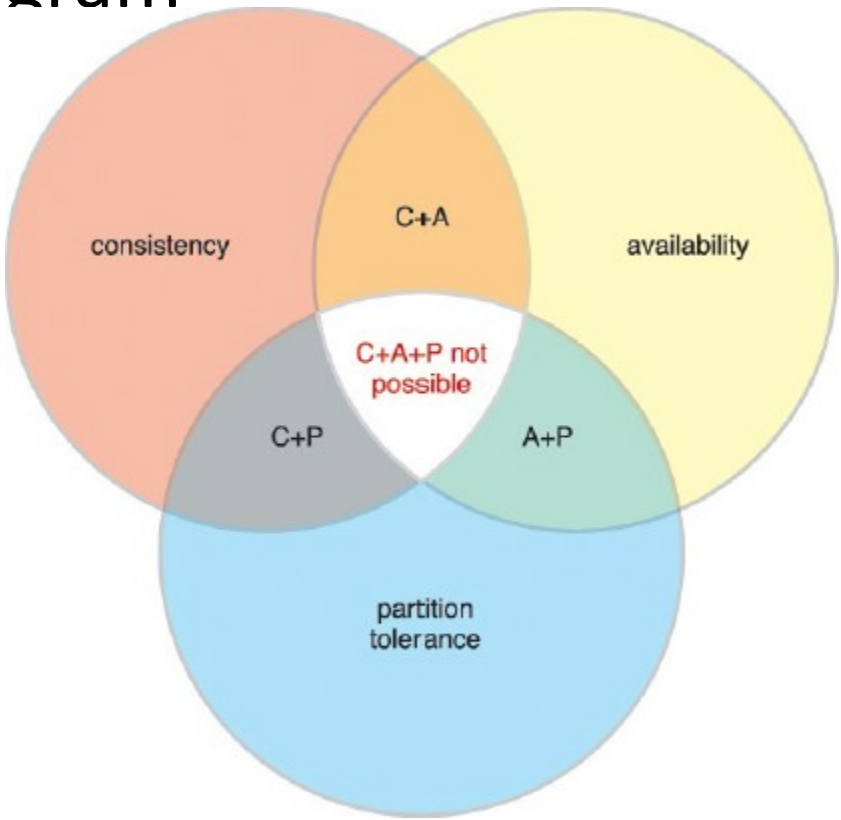


Availability and Partitioning

- In the event of a communication failure, requests from both users are still serviced (1, 2).
- However, with User B, the update fails as the record with id = 3 has not been copied over to Peer C because of network failure. The user is duly notified (3) that the update has failed.

CAP

- Venn Diagram



CAP Theorem

- In a distributed database, scalability and fault tolerance can be improved through additional nodes, although this challenges consistency (C). The addition of nodes can also cause availability (A) to suffer due to the latency caused by increased communication between nodes.
- Although communication outages are rare and temporary, partition tolerance (P) must always be supported by a distributed database. So a cluster has to be partition tolerant. It means C v/s A.
- Therefore, CAP is generally a choice between choosing either C+P or A+P. The requirements of the system will dictate which is chosen.

ACID property

- Transaction management property of RDBMS.
- Pessimist style of maintaining consistency by locking records
- Stands for
  - Atomicity
  - Consistency
  - Isolation
  - Durability

Atomicity

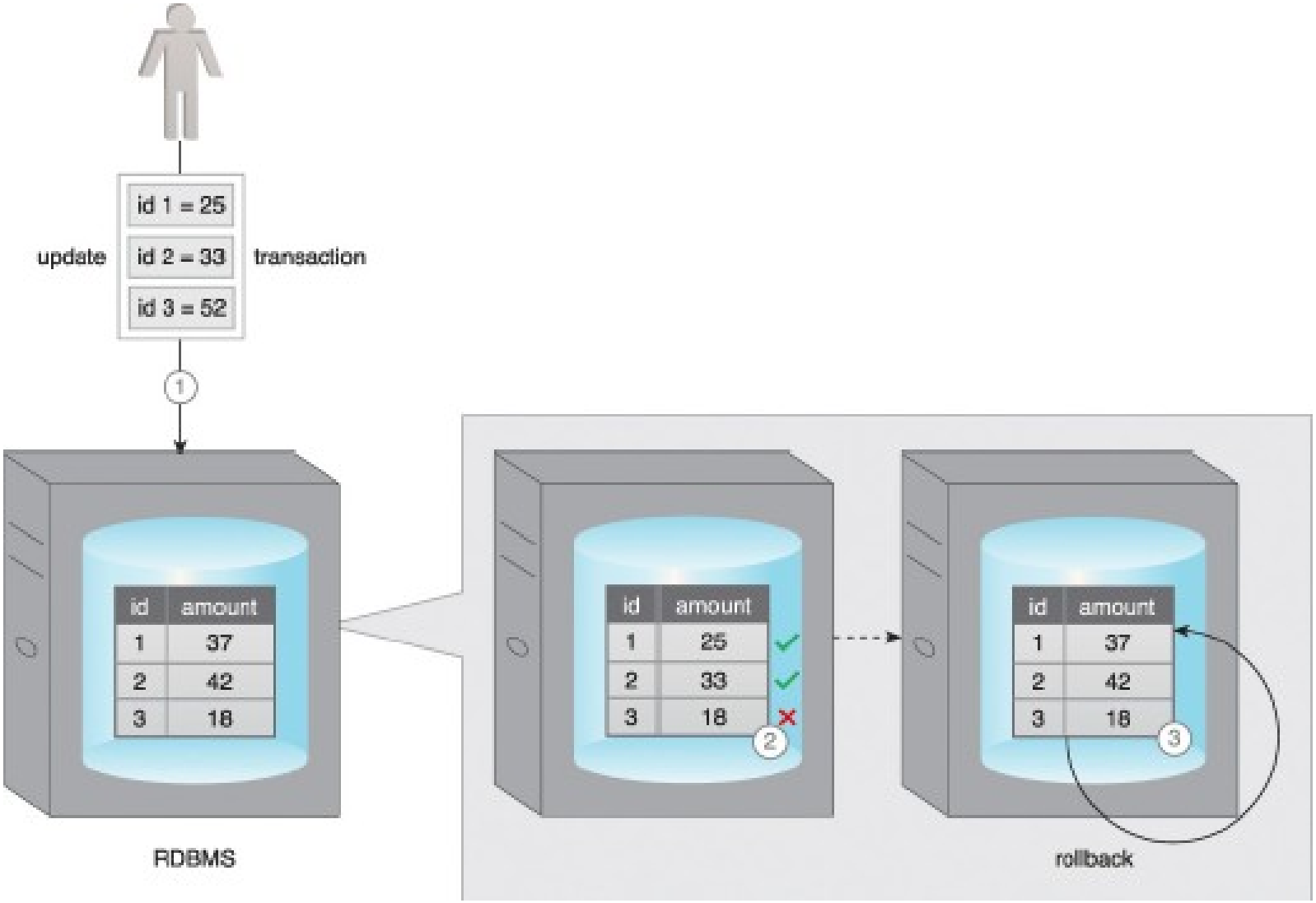
- Atomicity ensures that all operations will always succeed or fail completely. In other words, there are no partial transactions.
- The following steps are illustrated in Figure :

1. A user attempts to update three records as a part of a transaction.

2. Two records are successfully updated before the occurrence of an error.

3. As a result, the database roll backs any partial effects of the transaction and puts the system back to its prior state.

Atomicity

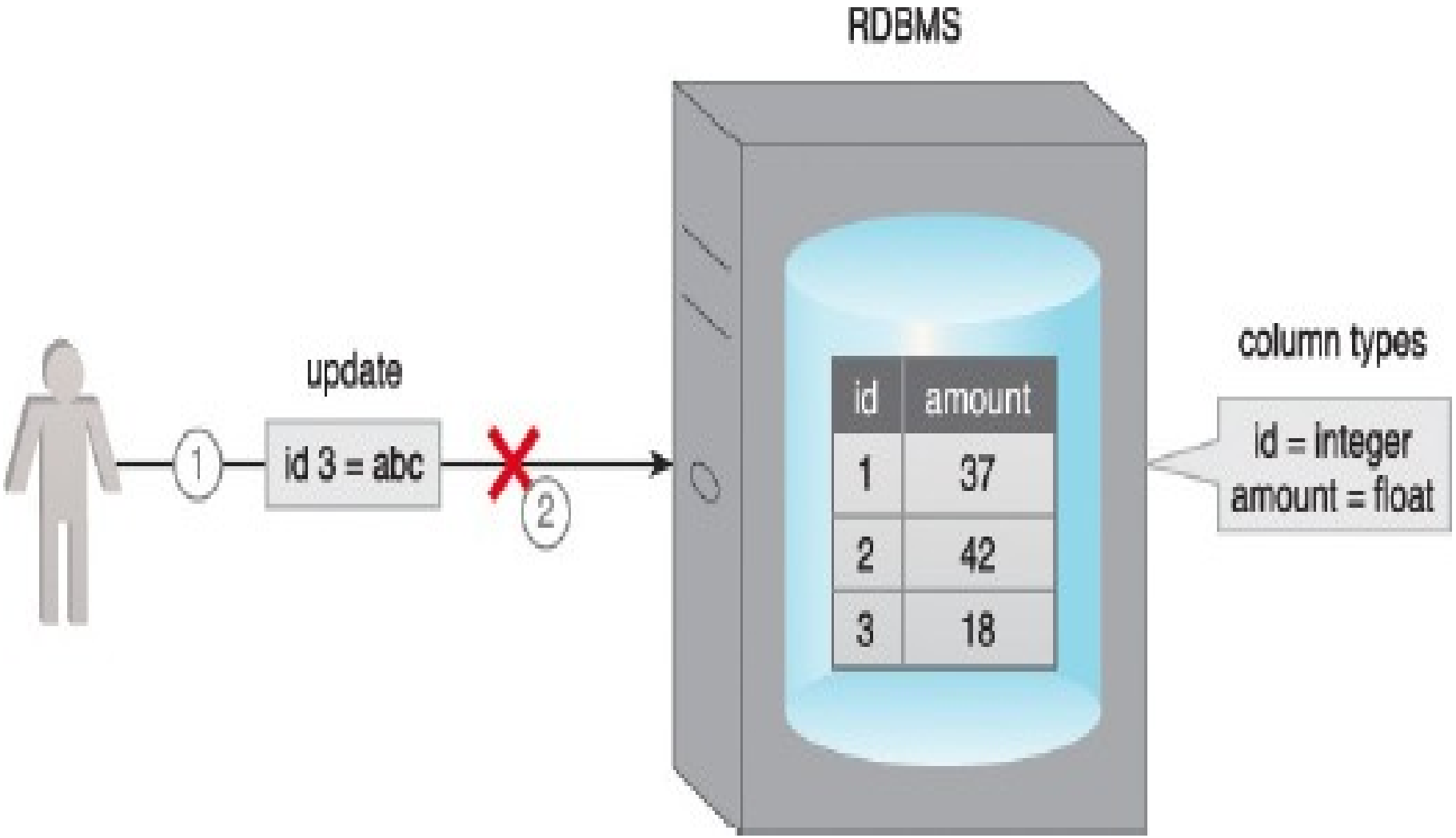

update
id 1 = 25
id 2 = 33
id 3 = 52
transaction
1
id amount
1 37
2 42
3 18
RDBMS
id amount
1 25
2 33
3 18
2
id amount
1 37
2 42
3 18
3
rollback

Consistency

- Consistency ensures that the database will always remain in a consistent state by ensuring that only data that conforms to the constraints of the database schema can be written to the database. Thus a database that is in a consistent state will remain in a consistent state following a successful transaction.

1. **A user attempts to update the amount column of the table that is of type float with a varchar value.**
2. **The database applies its validation check and rejects this update because the value violates the constraint checks for the amount column.**

Consistency



Isolation

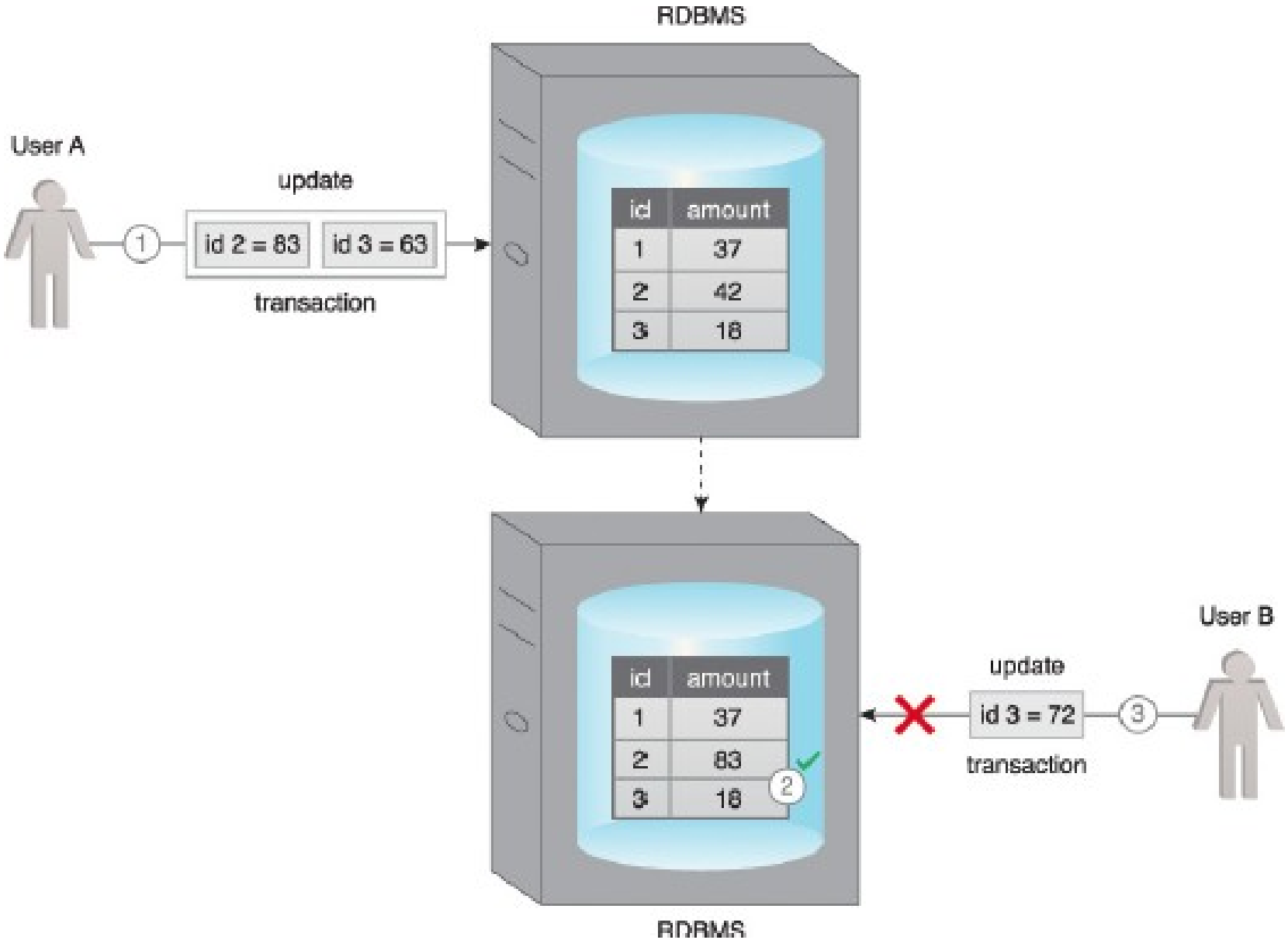
- Isolation ensures that the results of a transaction are not visible to other operations until it is complete.

1. User A attempts to update two records as part of a transaction.

2. The database successfully updates the first record.

3. However, before it can update the second record, User B attempts to update the same record. The database does not permit User B's update until User A's update succeeds or fails in full. This occurs because the record with id3 is locked by the database until the transaction is complete.

Isolation


RDBMS
User A
update
1
id 2 = 83
id 3 = 63
transaction
id amount
1 37
2 42
3 18
id amount
1 37
2 83
3 18
2
User B
update
id 3 = 72
3
transaction
RDBMS

Durability

- Durability ensures that the results of an operation are permanent. In other words, once a transaction has been committed, it cannot be rolled back. This is irrespective of any system failure.

1. A user updates a record as part of a transaction.

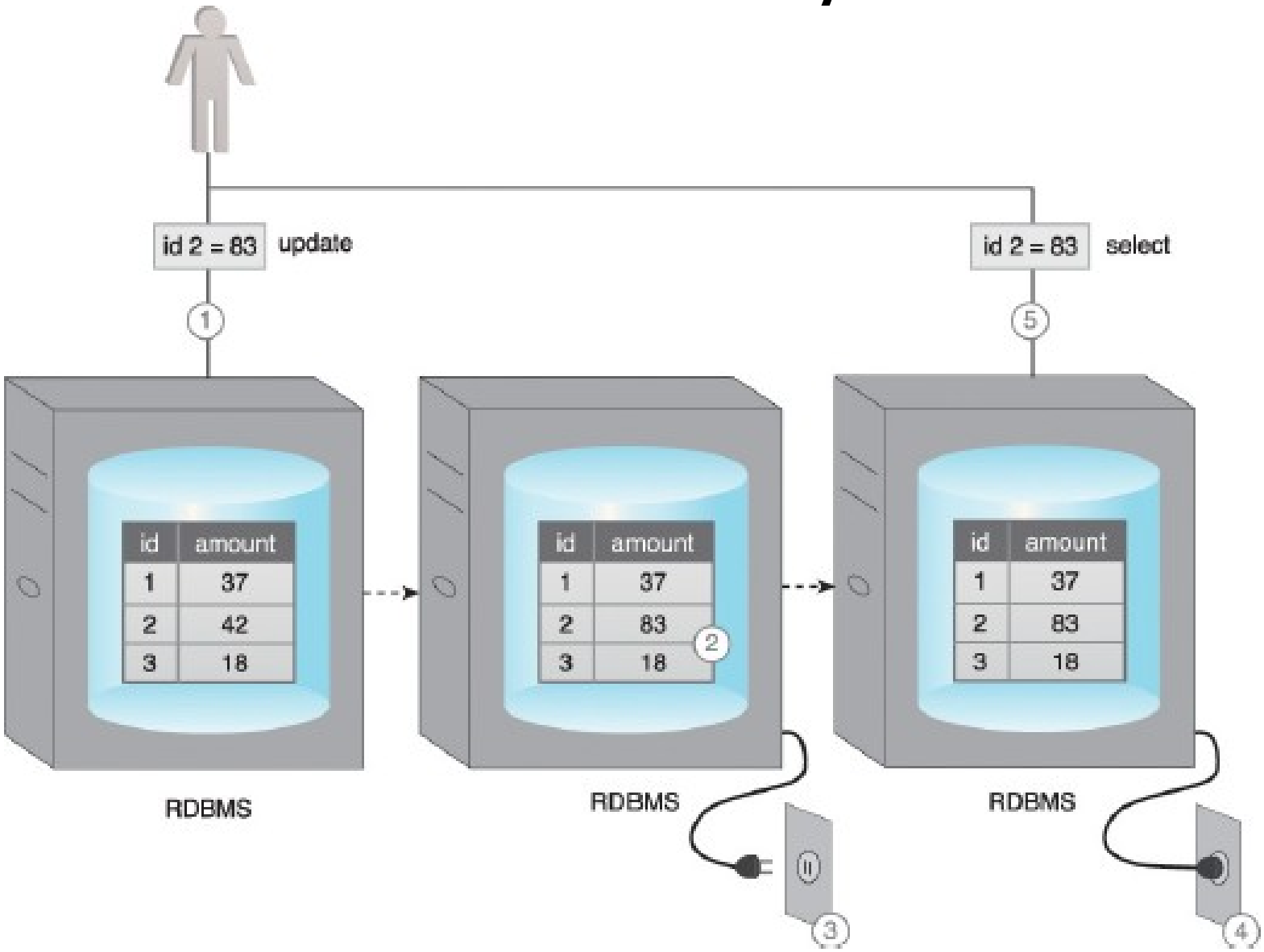
2. The database successfully updates the record.

3. Right after this update, a power failure occurs. The database maintains its state while there is no power.

4. The power is resumed.

5. The database serves the record as per last update when requested by the user.

Durability


id 2 = 83
update
id 2 = 83
select
1
5
id amount
1 37
2 42
3 18
RDBMS
id amount
1 37
2 83
3 18
2
RDBMS
3
id amount
1 37
2 83
3 18
RDBMS
4

ACID principle

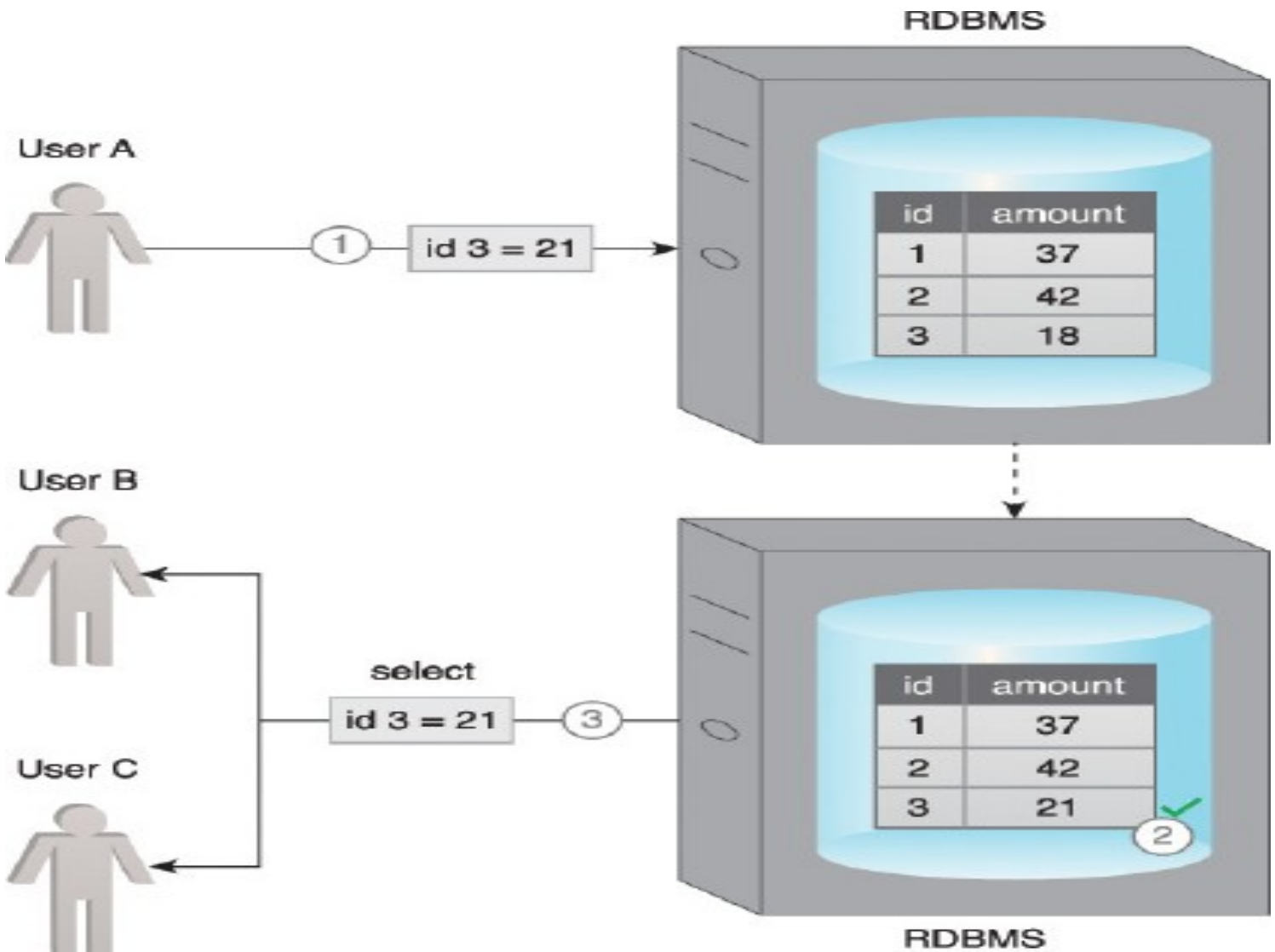
- **ACID principle results in consistent database behaviour**

1. User A attempts to update a record as part of a transaction.

2. The database validates the value and the update is successfully applied.

3. After the successful completion of the transaction, when Users B and C request the same record, the database provides the updated value to both the users.

ACID principle



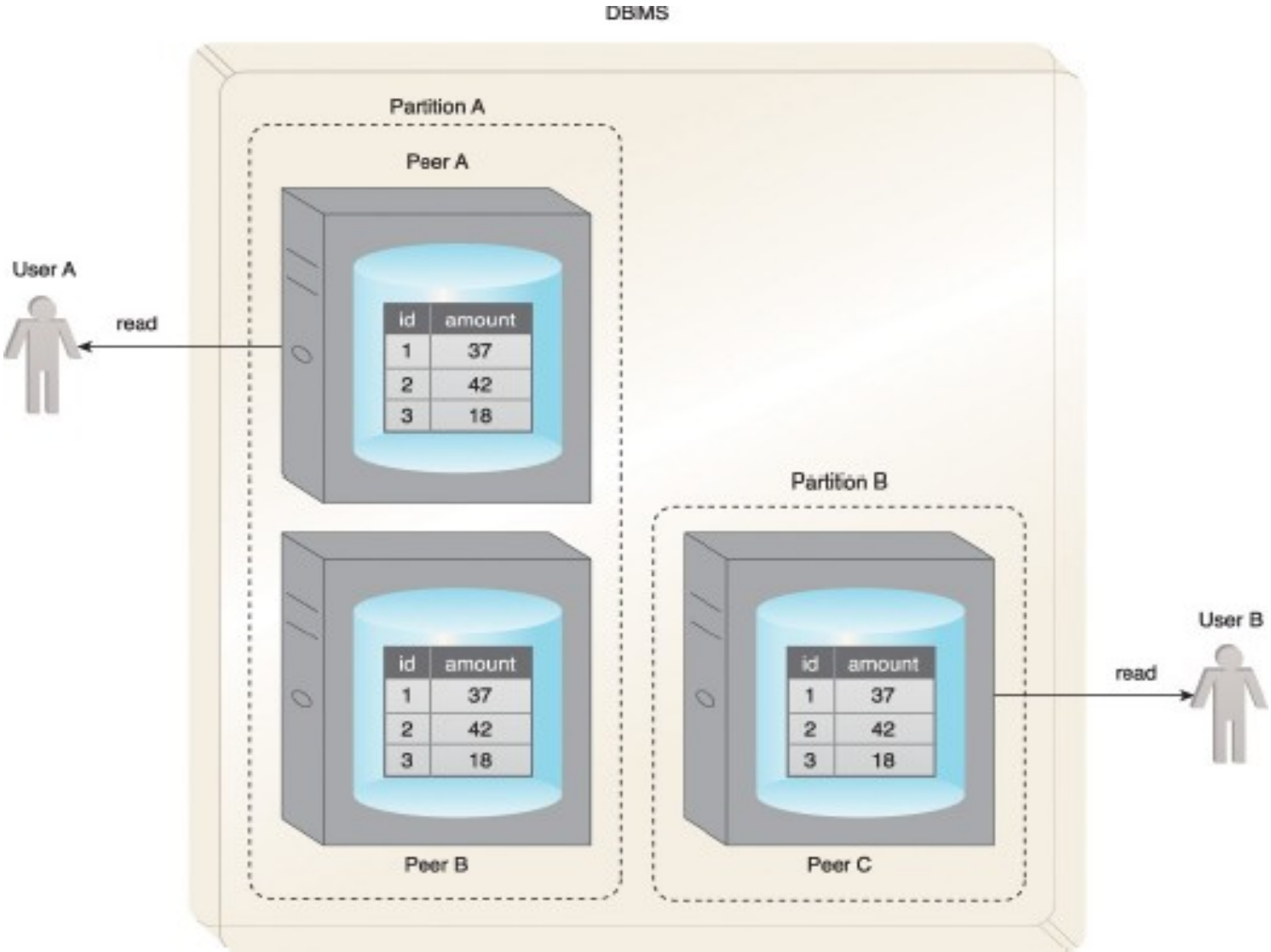
BASE

- BASE is a database design principle based on the CAP theorem and leveraged by database systems that use distributed technology. BASE stands for:
- **basically available**
- **soft state**
- **eventual consistency**
- When a database supports BASE, it favors availability over consistency. In other words, the database is A+P from a CAP perspective.
- In essence, BASE leverages optimistic concurrency by relaxing the strong consistency constraints mandated by the ACID properties.

BASE – basically available

- If a database is “basically available,” that database will always acknowledge a client’s request, either in the form of the requested data or a success/failure notification.
- The database is basically available, even though it has been partitioned as a result of a network failure.

User A and User B receive data despite the database being partitioned by a network failure.



BASE – soft state

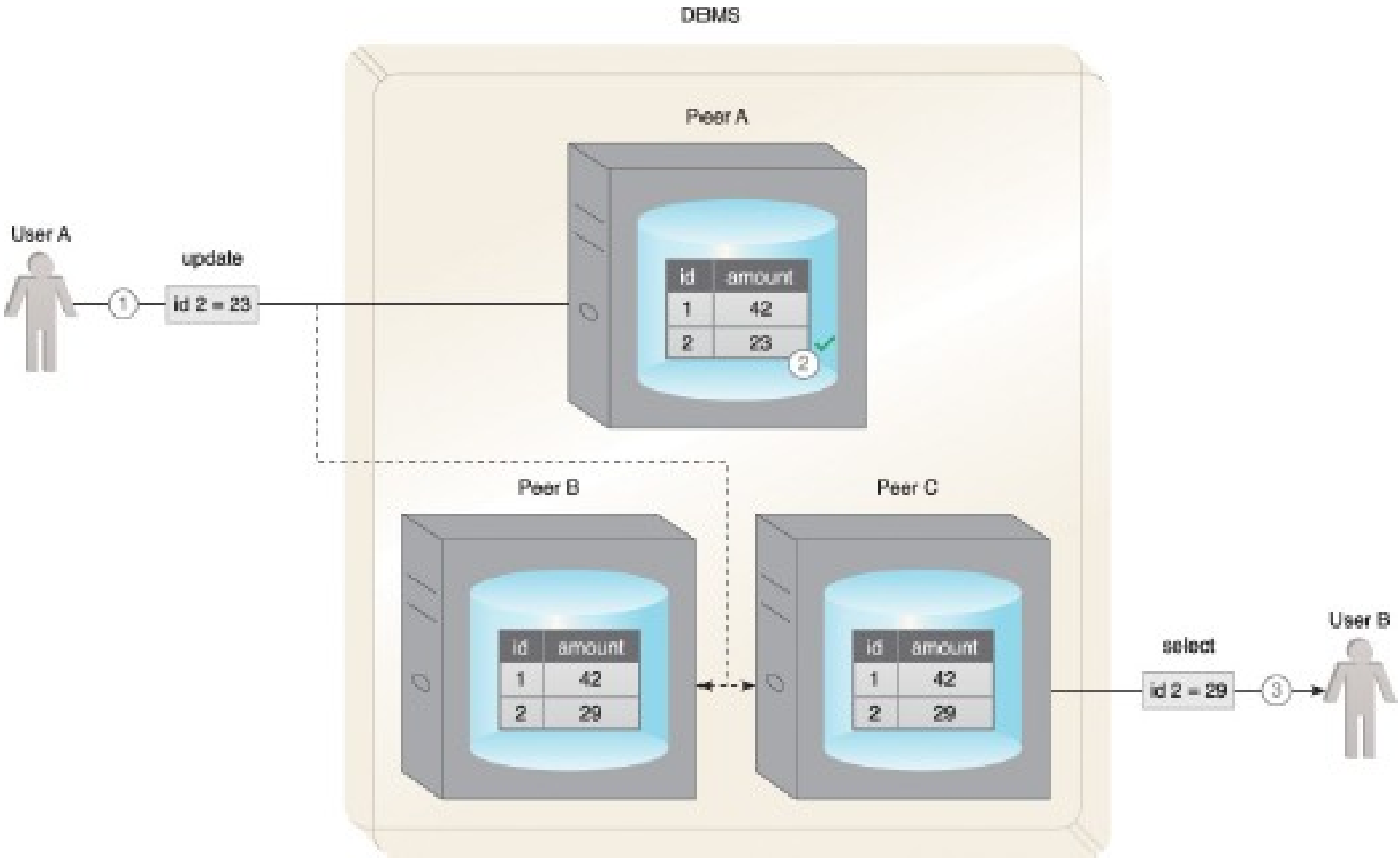
- Soft state means that a database may be in an inconsistent state when data is read; thus, the results may change if the same data is requested again.
- This is because the data could be updated for consistency, even though no user has written to the database between the two reads. This property is closely related to eventual consistency.

1. User A updates a record on Peer A.

2. Before the other peers are updated, User B requests the same record from Peer C.

3. The database is now in a soft state, and stale data is returned to User B.

An example of the soft state property of BASE is shown here.



BASE – Eventual consistency

- **Eventual consistency is the state in which reads by different clients, immediately following a write to the database, may not return consistent results. The database only attains consistency once the changes have been propagated to all nodes. While the database is in the process of attaining the state of eventual consistency, it will be in a soft state.**

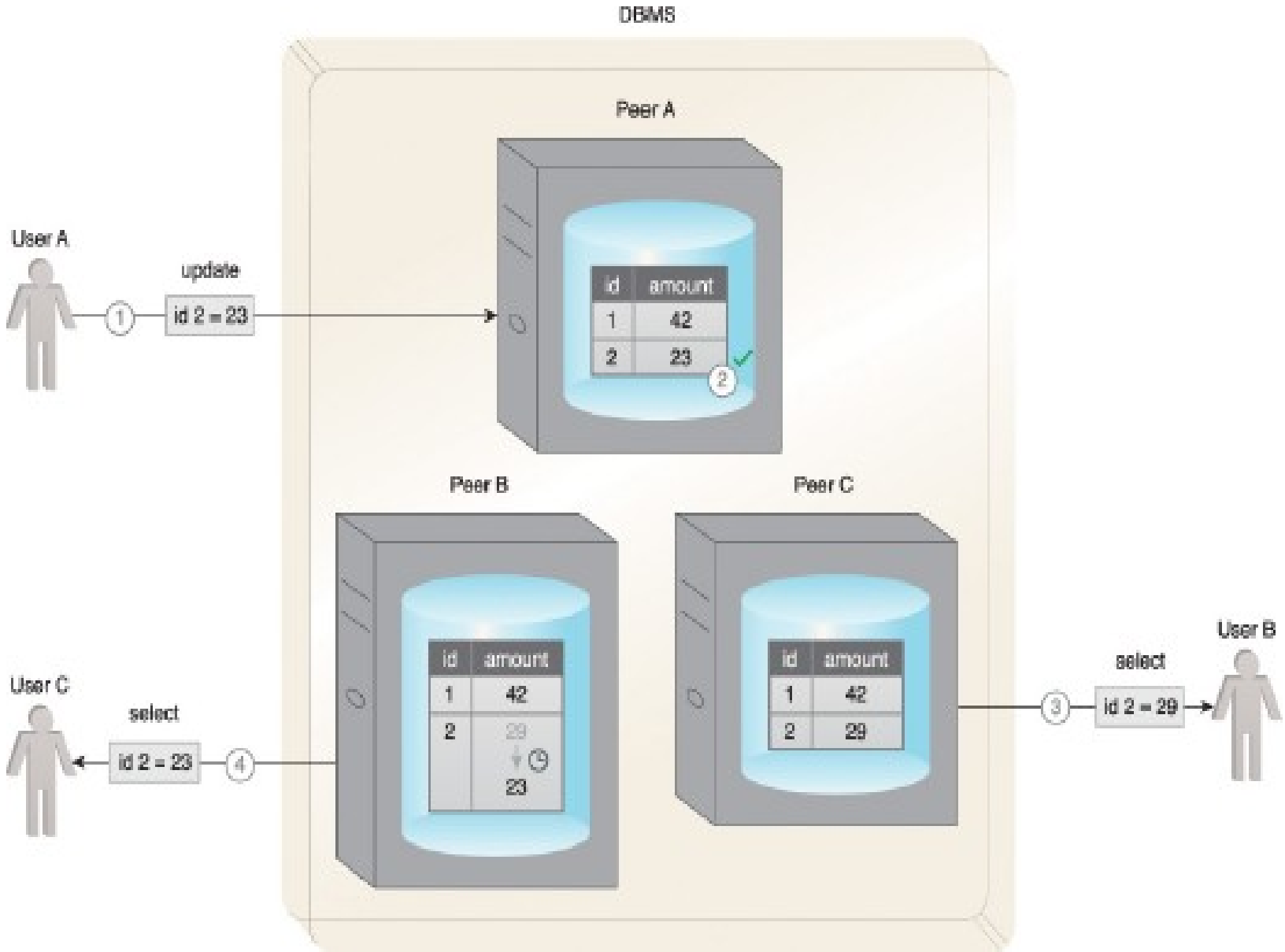
1. User A updates a record.

2. The record only gets updated at Peer A, but before the other peers can be updated, User B requests the same record.

3. The database is now in a soft state. Stale data is returned to User B from Peer C.

4. However, the consistency is eventually attained, and User C gets the correct value.

An example of the eventual consistency property of BASE.



BASE

- BASE emphasizes availability over immediate consistency, in contrast to ACID, which ensures immediate consistency at the expense of availability due to record locking.
- This soft approach toward consistency allows BASE compliant databases to serve multiple clients without any latency albeit serving inconsistent results.
- However, BASE-compliant databases are not useful for transactional systems where lack of consistency is a concern.